

LpWM: A Case for Sparse Representations in World Models

Yilun Kuang*
NYU, AMI Labs

Yash Dagade
Duke University

Quentin Le Lidec
NYU, AMI Labs

Lucas Maes
Mila, AMI Labs

Randall Balestriero
Brown University, AMI Labs

Yann LeCun
NYU, AMI Labs

Abstract

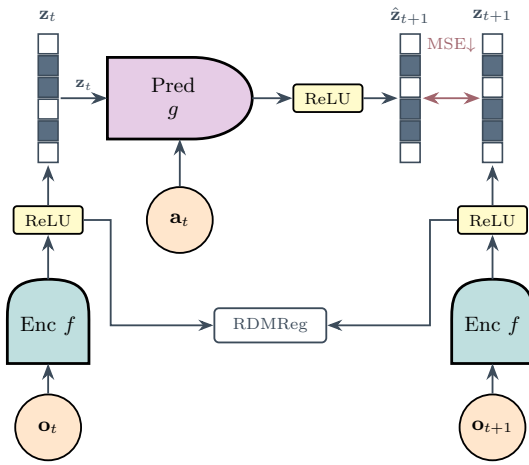
Joint-embedding predictive architectures (JEPAs) learn latent dynamics for planning and avoid representation collapse by matching features to maximum-entropy distributions such as isotropic Gaussians, yielding dense representations. However, it is unclear whether dense representations are the most favorable geometry for modeling dynamics. In this work, we ask whether a different geometry—sparse representations—can make action-conditioned latent dynamics easier to model, and what dynamical structure emerges from such representations. We first show that nonlinear Lipschitz dynamics can be approximated arbitrarily well by action-conditioned linear dynamics in a sufficiently high-dimensional one-hot latent space, with rollout error vanishing as the dimension grows. This motivates distributed sparse representations as a practical relaxation of one-hot sparsity. We introduce LpWorldModel (LpWM), a JEPA model regularized with Rectified Distribution Matching Regularization (RDMReg) to match encoder features to a Rectified Generalized Gaussian distribution, yielding non-negative sparse codes. Empirically, sparsity lowers the predictor complexity required for successful planning: on PushT, sparse LpWM outperforms dense LeWM by up to 57% in planning success at intermediate predictor capacities. This advantage also extends beyond Gaussian distribution matching, with LpWM outperforming dense VICReg representations across multiple predictor families. We further find that the learned sparse representations are mode-factored, with support encoding discrete dynamical regimes and feature magnitudes capturing continuous within-regime state. Together, these results suggest that sparse representations can reduce the predictor complexity required for control while revealing interpretable structure.

1 Introduction

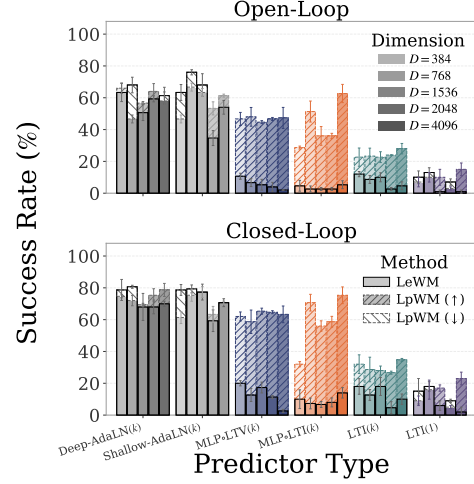
Joint-embedding predictive architectures (JEPA) have emerged as a practical recipe for model-based control from pixels: an encoder maps observations to a latent space, a predictor rollouts the latent forward under actions, and planning is carried out by model-predictive control (MPC) in latent space [LeCun et al., 2022]. In the case of end-to-end training, where the encoder and predictor are jointly trained, an anti-collapse loss is necessary to prevent the feature representation from degenerating to a constant or low-rank subspaces [Jing et al., 2021].

Prior methods such as PLDM [Sobal et al., 2025] use VReg [Bardes et al., 2022], while LeWM [Maes et al., 2026b] uses SIGReg [Balestriero and LeCun, 2025] to prevent collapse through information maximization, yielding dense representations in which all latent coordinates are nonzero almost

*Correspondence to: Yilun Kuang <yilun.kuang@nyu.edu>, Yann LeCun <yann.lecun@nyu.edu>. **Code:** <https://github.com/YilunKuang/lpworldmodel>.



(a) LpWM Schematic Diagram



(b) PushT Planning Performance.

Figure 1: **LpWorldModel: a JEPA model with sparse representations.** (a) An encoder f maps observation $\mathbf{o}_t$ to non-negative, sparse latents $\mathbf{z}_t$ via a terminal (Rep)ReLU function. The predictor g takes inputs $\mathbf{z}_t$ and actions $\mathbf{a}_t$ to predict $\hat{\mathbf{z}}_{t+1}$, compared against the encoded target $\mathbf{z}_{t+1} = f(\mathbf{o}_{t+1})$. RDMReg [Kuang et al., 2026b] matches per-timestep latent marginals to a Rectified Generalized Gaussian, preventing collapse and inducing sparsity. (b). Arrows indicate the success-rate difference between LpWM and LeWM: $\uparrow$ denotes higher performance for LpWM, while $\downarrow$ denotes lower performance. LpWM outperforms LeWM with more linear predictor variants (MLP $\circ$ LTV(k), MLP $\circ$ LTI(k), and LTI(k)). See full specifications in Table 1) by up to 57% in success rate, while performing on par with LeWM using the high-capacity DiT predictors. This suggests that sparse representations induce a latent space with simpler dynamics to learn compared to their dense counterparts.

surely. While effective for preventing collapse, it remains unclear whether dense features are the most favorable geometry for modeling controlled dynamics.

In this work, we ask whether sparse representations provide a more favorable geometry for modeling action-conditioned dynamics. Our motivation comes from an idealized limit: for Lipschitz controlled dynamics on a compact state space, sufficiently high-dimensional one-hot representations can approximate the dynamics with action-conditioned linear latent dynamics (Proposition 1), with rollout error vanishing as the dimension grows (Corollary 1). While exact one-hot codes are impractical, this suggests distributed sparse representations, in which a nontrivial fraction of latent coordinates are exactly zero, as a learnable relaxation that may simplify the dynamics.

We introduce LpWorldModel (**LpWM**), a JEPA world model regularized with RDMReg [Kuang et al., 2026b] to learn non-negative, sparse codes. RDMReg matches rectified feature distributions to Rectified Generalized Gaussian targets, whose underlying Generalized Gaussian distributions are maximum-entropy under expected ℓ_p -norm constraints with $p \leq 1$ yields sparse targets.

Empirically, sparsity lowers the predictor complexity required for successful planning: predictors that fail over dense codes succeed over sparse ones. The resulting representations are also mode-factored: their support encodes discrete dynamics regimes, while feature magnitudes encode continuous within-regime state. Together, these results suggest that sparse latent geometry can make dynamics both easier to model and more interpretable.

Contributions.

- **LpWorldModel.** We introduce LpWM, a JEPA world model that uses RDMReg to learn non-negative sparse latent representations for latent dynamics modeling and planning.
- **Sparse representations for latent linearization.** We formalize the intuition that sufficiently high-dimensional one-hot representations can approximate nonlinear controlled dynamics with linear latent transitions, motivating distributed sparse codes as a practical relaxation.

- **Lower-complexity latent dynamics.** We show empirically that sparsity reduces the predictor capacity needed for successful planning: while Wall dynamics are already solved by linear predictors, on the harder PushT environment sparse LpWM substantially outperforms dense LeWM with intermediate-capacity predictors, by up to 57% in success rate.
- **Mode-factored latent dynamics.** We find that sparse representations organize dynamics into interpretable factors, with support encoding discrete dynamical regimes and feature magnitudes capturing continuous within-regime state.

2 Method

2.1 LpWM: JEPA with Rectified Distribution Matching Regularization (RDMReg)

Let $\mathbf{z}_{t+1} = f_{\theta}(\mathbf{x}_{t+1}) \in \mathbb{R}^d$ be the encoder representation of the observation $\mathbf{x}_{t+1}$ and $\hat{\mathbf{z}}_{t+1} = g_{\phi}(\mathbf{z}_t, \mathbf{a}_t)$ be the predictor output with action inputs $\mathbf{a}_t \in \mathbb{R}^{d_a}$. The action-conditioned JEPA uses the following loss function to jointly train the encoder f_{θ} and the predictor g_{ϕ} :

$$\min_{\theta, \phi} \|\hat{\mathbf{z}}_{t+1} - \mathbf{z}_{t+1}\|_2 + \lambda_{\text{RDMReg}} \cdot \mathcal{R}(\mathbf{z}_{t+1}) \quad (1)$$

where λ_{RDMReg} is a hyperparameter and $\mathcal{R}(\cdot)$ is an anti-collapse regularizer. We focus on the setting where $\mathcal{R}(\cdot)$ is the Rectified Distribution Matching Regularization (RDMReg) [Kuang et al., 2026b]:

$$\mathcal{R}(\mathbf{z}) = \mathbb{E}_{\mathbf{c} \sim \text{Unif}(\mathbb{S}_{\ell_2}^{d-1})} [\mathcal{L}(\mathbb{P}_{\mathbf{c}^\top \mathbf{z}} \|\mathbb{P}_{\mathbf{c}^\top \mathbf{y}})] \quad (2)$$

where the random projection vector $\mathbf{c} \in \mathbb{R}^d$ is sampled uniformly from the ℓ_2 unit sphere $\mathbb{S}_{\ell_2}^{d-1} := \{\mathbf{x} \in \mathbb{R}^d \mid \|\mathbf{x}\|_2 = 1\}$, the target random vector $\mathbf{y} \sim \prod_{i=1}^d \text{ReLU}(\mathcal{GN}_p(\mu, \sigma))$ follows the Rectified Generalized Gaussian (RGG) distribution, $\mathbb{P}_x$ denotes the probability distribution for the random variable x , and $\mathcal{L}(\cdot \|\cdot)$ is a loss function that compute discrepancies between probability distributions. In our case, we follow Kuang et al. [2026b] and set $\mathcal{L}(\cdot \|\cdot)$ to be the 2-Wasserstein distance. By default we choose $\mu = 0$, $\sigma = \sqrt{1/2}$, and $p = 1$ and hence our target distribution reduces to the Rectified Laplace distribution [Kuang et al., 2026b]. Note that when $\mu = 0$, $\sigma = 1$, $p = 2$, and in the absence of ReLU, we recover dense isotropic Gaussian used in LeWM [Maes et al., 2026b].

2.2 Architecture and Rectification

Following Maes et al. [2026b], our encoder f_{θ} is a ViT [Dosovitskiy et al., 2021] with the output CLS token passed through a three-layer MLP projector to produce the embedding $\mathbf{z} \in \mathbb{R}^d$. The predictor g_{ϕ} is a Transformer with AdaLN-zero action conditioning [Peebles and Xie, 2023] and a final three-layer MLP projector that predicts the next embedding $\hat{\mathbf{z}}_{t+1}$.

To induce exact zeros, both encoder and predictor MLP projector outputs pass through a reparameterized ReLU [Wang et al., 2024, Kuang et al., 2026b]: $\text{RepReLU}(x) = \text{sg}(\text{ReLU}(x)) + \text{GeLU}(x) - \text{sg}(\text{GeLU}(x))$, where $\text{sg}(\cdot)$ denotes the stop-gradient operator. In the forward pass, $\text{RepReLU}(x) = \text{ReLU}(x)$, yielding exact zeros, while in the backward pass gradients flow through $\text{GeLU}(x)$ to mitigate the dying-ReLU pathology [Hendrycks and Gimpel, 2023]. Importantly, LpWM does not rely on stop-gradient for collapse prevention: standard ReLU with RDMReg trains without collapse and yields exactly sparse representations [Kuang et al., 2026b]. We therefore use RepReLU as an optimization safeguard against dead neurons rather than as an essential component of the method. See Figure 1a for LpWM architecture.

2.3 Planning with the Learned World Model

At test time the world model serves as the dynamics model for model-predictive control (MPC): we encode the current and goal observation images to $\mathbf{z}_0$ and $\mathbf{z}_g$, and search for an action sequence whose predicted latent rollout minimizes the goal cost $\mathcal{C} = \|\hat{\mathbf{z}}_T - \mathbf{z}_g\|_2$ using the Cross-Entropy Method (CEM) [Rubinstein, 1999, Zhou et al., 2025]. See additional experimental details in Appendix D.

3 Does Sparsity Simplify Latent Dynamics?

In the following sections, we ask whether a sparse code makes the action-conditioned latent dynamics simpler by requiring a less expressive predictor than a dense code. We first show that a sufficiently

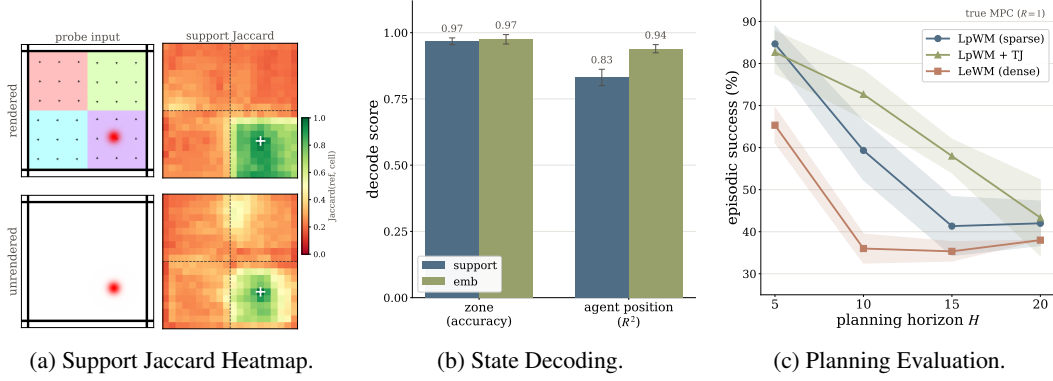


Figure 2: **The sparse support recovers the piecewise structure of the dynamics.** (a) Support Jaccard heatmap over a 20×20 grid: the Jaccard index of each cell’s support with a fixed reference cell (white +) is high within the reference zone and drops across zone boundaries (dashed), for a model trained with rendered zones (top) and one with no visual cues (bottom). (b) The binary support decodes the ground-truth zone identity as accurately as the full embedding, while agent position is decoded better from the magnitudes than from the support. (c) Planning success vs. horizon (closed-loop, 3-seed mean $\pm$ std). Goal locations of the dot agent are sampled randomly. Both sparse variants beat dense LeWM at every horizon.

high-dimensional sparse code renders the latent dynamics exactly linear under appropriate assumptions (Proposition 1). We then relax this to a learnable distributed sparse code and test, along a predictor-complexity ladder (Table 1), whether a simpler predictor plans better under sparsity.

3.1 Linearization of Nonlinear Dynamics with Sparse Representations

In Proposition 1, we show that compact controlled systems with Lipschitz dynamics admit finite one-hot sparse encodings with exactly linear latent dynamics. This is closely related to classical symbolic abstractions based on state-space quantization [Pola et al., 2008], which we reinterpret as one-hot sparse latent codes.

Proposition 1 (Linear Approximation of Nonlinear Dynamics with Sparse Representations). *Let $\mathcal{X} \subset \mathbb{R}^d$ be a compact state space and let $\mathcal{A} \subset \mathbb{R}^p$ be a compact continuous action space. Consider a deterministic discrete-time controlled dynamical system $\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{a}_t)$ where $f : \mathcal{X} \times \mathcal{A} \rightarrow \mathcal{X}$. Assume that f is uniformly Lipschitz in the state variable, i.e. there exists a finite constant $L \geq 0$ such that $\|f(\mathbf{x}, \mathbf{a}) - f(\mathbf{y}, \mathbf{a})\| \leq L\|\mathbf{x} - \mathbf{y}\|$ for every $\mathbf{x}, \mathbf{y} \in \mathcal{X}$ and $\mathbf{a} \in \mathcal{A}$. Then, for every $\varepsilon > 0$, there exist 1) a finite latent dimension N ; 2) a one-hot sparse encoder $E_\varepsilon : \mathcal{X} \rightarrow \mathbb{R}^N$; 3) a decoder $D_\varepsilon : \{\mathbf{e}_1, \dots, \mathbf{e}_N\} \rightarrow \mathcal{X}$; and 4) an action-conditioned family of matrices $P_\varepsilon : \mathcal{A} \rightarrow \{0, 1\}^{N \times N}$, $\mathbf{a} \mapsto P_\varepsilon(\mathbf{a})$, such that the latent dynamics $\mathbf{z}_{t+1} = P_\varepsilon(\mathbf{a}_t)\mathbf{z}_t$ are exactly linear in $\mathbf{z}_t$, and the decoded one-step prediction satisfies*

$$\sup_{\mathbf{x}_t \in \mathcal{X}, \mathbf{a}_t \in \mathcal{A}} \|f(\mathbf{x}_t, \mathbf{a}_t) - D_\varepsilon(P_\varepsilon(\mathbf{a}_t)E_\varepsilon(\mathbf{x}_t))\| \leq (L + 1)\varepsilon. \quad (3)$$

Proof. See Appendix G.1 □

In Corollary 1, we consider one example of a compact domain and show that the rollout error with respect to the ground-truth dynamics vanishes as the encoding dimension grows.

Corollary 1 (Sparsity in High-Dimensions). *Let the state space be $\mathcal{X} = [0, 1]^d$ and consider dividing each coordinate into n equal intervals, producing $N = n^d$ grid cells. We represent each cell by its center with the covering radius $\varepsilon_N = \frac{\sqrt{d}}{2}N^{-1/d}$. Hence the one-step prediction error satisfies*

$$\sup_{\mathbf{x}, \mathbf{a}} \|f(\mathbf{x}, \mathbf{a}) - D_{\varepsilon_N}(P_{\varepsilon_N}(\mathbf{a})E_{\varepsilon_N}(\mathbf{x}))\| \leq \frac{\sqrt{d}}{2}(L + 1)N^{-1/d} = O(N^{-1/d}). \quad (4)$$

For any fixed rollout horizon H , $\|\mathbf{x}_H - \hat{\mathbf{x}}_H\| \leq \frac{\sqrt{d}}{2}N^{-1/d} \sum_{k=0}^H L^k$ and $\lim_{N \rightarrow \infty} \|\mathbf{x}_H - \hat{\mathbf{x}}_H\| = 0$.

Table 1: **Predictors with Different Levels of Complexity.** $\hat{\mathbf{z}}_{t+1}$ is the predicted latent, $\mathbf{z}_{t-i}$ past latents, and $\mathbf{a}_t$ the action. AdaLN predictors are Transformer-based models with Adaptive Layer Normalization for action conditioning, i.e. DiT architecture [Peebles and Xie, 2023]. The Deep-AdaLN(k) predictor is the one used in LeWM [Maes et al., 2026b]. LTI denotes a linear time-invariant predictor with fixed operators $\mathbf{A}_i, \mathbf{B}$, while LTV denotes a linear time-varying predictor with state-dependent operators $\mathbf{A}_i(\mathbf{z}_t), \mathbf{B}(\mathbf{z}_t)$. We use MLP $\circ$ to denote one extra nonlinear transform. The history k denotes context length, with 1 the single-frame case. The output link is $\sigma = \text{identity}$ for dense LeWM and $\sigma = \text{RepReLU}$ for sparse LpWM. See Appendix B for details.

Predictor	Layers	History (k)	Functional form
Deep-AdaLN(k)	6	3	$\hat{\mathbf{z}}_{t+1} = \sigma \left(\text{AdaLN}^{(6)}(\mathbf{z}_{t-k+1:t}, \mathbf{a}_t) \right)$
Shallow-AdaLN(k)	1	3	$\hat{\mathbf{z}}_{t+1} = \sigma \left(\text{AdaLN}^{(1)}(\mathbf{z}_{t-k+1:t}, \mathbf{a}_t) \right)$
MLP$\circ$LTV(k)	1	3	$\hat{\mathbf{z}}_{t+1} = \sigma \left(\mathbf{W} \text{ReLU} \left(\sum_{i=0}^{k-1} \mathbf{A}_i(\mathbf{z}_t) \mathbf{z}_{t-i} + \mathbf{B}(\mathbf{z}_t) \mathbf{a}_t \right) \right)$
MLP$\circ$LTI(k)	1	3	$\hat{\mathbf{z}}_{t+1} = \sigma \left(\mathbf{W} \text{ReLU} \left(\sum_{i=0}^{k-1} \mathbf{A}_i \mathbf{z}_{t-i} + \mathbf{B} \mathbf{a}_t \right) \right)$
LTI(k)	1	3	$\hat{\mathbf{z}}_{t+1} = \sigma \left(\sum_{i=0}^{k-1} \mathbf{A}_i \mathbf{z}_{t-i} + \mathbf{B} \mathbf{a}_t \right)$
LTI(1)	1	1	$\hat{\mathbf{z}}_{t+1} = \sigma (\mathbf{A} \mathbf{z}_t + \mathbf{B} \mathbf{a}_t)$

Proof. See Appendix G.2. □

We note that the approximation error of one-step prediction under an exact one-hot sparse encoding decreases as $O(N^{-1/d})$, and therefore suffers from the curse of dimensionality. Nonetheless, Corollary 1 illustrates a useful trade-off: higher-dimensional sparse representations can simplify the latent transition map, motivating distributed sparse codes as a practical relaxation that may reduce the complexity of the learned dynamics model. In the following experiments, we test whether this simplification enables successful planning with less expressive predictors. In the special case where the learned latent dynamics reduce further to a standard linear controlled system, classical control methods such as LQR become directly applicable under standard assumptions [Kalman et al., 1960].

3.2 Experiment Designs

In practice, exact high-dimensional one-hot encodings are infeasible to optimize, and we need exponentially large feature dimensions to linearize the latent dynamics. LpWM instead learns distributed sparse codes whose sparsity is controlled by the target-distribution parameter μ . Although these codes maybe not linearize the dynamics exactly, we hypothesize that they make the latent dynamics simpler to model than dense representations, so that lower-capacity predictors can suffice.

To test this, we consider a list of predictor options in Table 1, ranging from nonlinear DiT predictors with AdaLN action conditioning [Peebles and Xie, 2023] to linear time-invariant (LTI) state-space models. We train sparse LpWM and dense LeWM with the same encoder architecture described in Section 2.2, but replace the standard DiT predictor with candidates in Table 1 across latent dimensions $D \in \{384, 768, 1536, 2048, 4096\}$ with their corresponding parameter counts in Table 2.

Additionally, we test a variant of LeWM in which SIGReg [Balestriero and LeCun, 2025] is replaced with VICReg [Bardes et al., 2022], yielding another JEPA model with dense representations. For simplicity, we use VICReg to denote this LeWM variant in Figure 3b.

Since our experiment design involves comparing models under width scaling, we use maximum update parameterization (μP) to stabilize training across widths, with details in Appendix F. Experiments are conducted on Wall and PushT environments (Appendix C).

3.3 Results

In Figure 3a, we observe that for the Wall environment, even the simplest linear LTI(1) predictor reaches nearly 100% closed-loop CEM success rate for both LeWM and LpWM, leaving little room to distinguish sparse and dense representations.

Table 2: **Predictor parameter counts across latent dimensions D .** Counts are for the predictor alone for each D used in our experiments. LTI, MLPoLTI, and MLPoLTV are attention-free and far smaller than the AdaLN Transformers.

Predictor	$D=384$	$D=768$	$D=1536$	$D=2048$	$D=4096$
Deep-AdaLN(k)	25.8M	62.2M	166.9M	260.2M	822.4M
Shallow-AdaLN(k)	5.6M	13.0M	33.1M	50.4M	151.1M
MLPoLTV(k)	0.81M	3.10M	12.1M	21.4M	84.7M
MLPoLTI(k)	0.74M	2.95M	11.8M	21.0M	83.9M
LTI(k)	0.59M	2.36M	9.44M	16.8M	67.1M
LTI(1)	0.30M	1.18M	4.72M	8.39M	33.6M

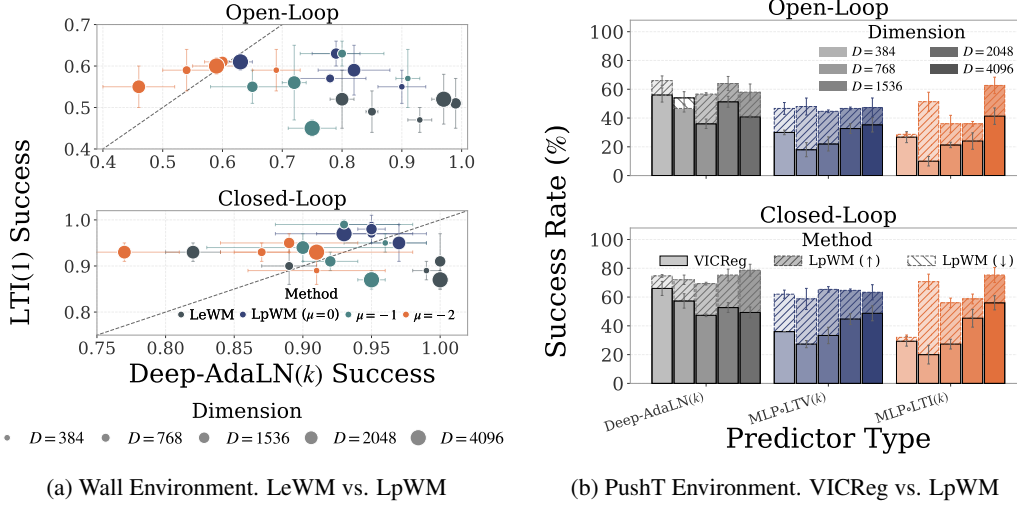


Figure 3: **Linear predictor is sufficient for the wall environment regardless of dense or sparse embeddings, and LpWM outperforms VICReg-based JEPa across predictor settings in PushT.**

(a) We plot the planning success rate of the linear LTI(1) predictor against the nonlinear Deep-AdaLN(k), where each dot denotes one setting of method and feature dimension. For simplicity, we use $\mu = *$ to denote LpWM with the target distribution parameter μ . We show the open-loop evaluation on top and closed-loop results at bottom. Both are ran using CEM planner. We observe that the closed-loop success rate for LTI(1) predictor using SIGReg is already approaching 100%, indicating that the wall environment can already be described well by a linear dynamics model in the latent space. (b) We have already shown LeWM vs. LpWM in Figure 1b. Here we compare LpWM with a variant of LeWM where we replaced SIGReg with VICReg in the PushT environment across predictor settings. As opposed to SIGReg, we observe consistent improvement from LpWM over VICReg even for the Deep-AdaLN(k) predictor, and the gap between sparse LpWM and dense methods (i.e. VICReg here) continues to hold for MLPoLTV(k) and MLPoLTI(k) predictors. See full hyperparameter sweeps in Appendix H.2.

We therefore turn to the harder PushT environment with results shown in Figure 1b. At the lowest predictor capacity, LTI(1) fails for both LeWM and LpWM. Thus PushT differs from Wall in that the underlying dynamics cannot be modelled well by a linear predictor in practice.

At the highest capacity, Deep-AdaLN(k) and Shallow-AdaLN(k) perform similarly for both representations, indicating that the predictor complexity saturates and we don’t observe significant advantages of sparse LpWM against dense LeWM.

The difference emerges at intermediate capacity: on PushT, sparse LpWM improves planning success over dense LeWM by 24%–57% with MLPoLTI(k), 36%–45% with MLPoLTV(k), and 11%–23% with LTI(k). These performance gaps are verified across extensive hyperparameter sweeps for both sparse LpWM and dense LeWM in Appendix H.1. We also report the ℓ_0 norms of the LpWM embeddings in Table 3.

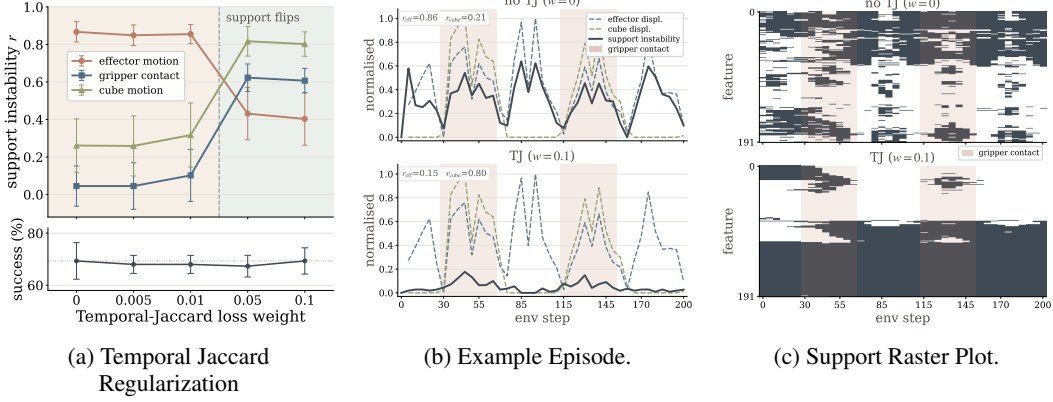


Figure 4: Temporal-Jaccard (TJ) makes the cube support track contact, not effector motion. (a) As TJ strength increases, support-instability correlation with effector motion drops ($0.87 \rightarrow 0.40$) while cube-motion ($0.26 \rightarrow 0.80$) and gripper-contact ($0.05 \rightarrow 0.61$) rise—at no change in planning success. (b) One example episode: without TJ (top) support instability tracks effector displacement ($r_{\text{eff}}=0.86$); with TJ (bottom) it tracks cube displacement ($r_{\text{cube}}=0.80$), spiking inside the shaded contact windows. (c) Support raster: without TJ the support rapidly transitions across frame; with TJ it is smoothly varying in time, reorganizing almost during the shaded contact window.

Additionally, Figure 3b shows that LpWM outperforms VICReg across Deep-AdaLN(k), MLPoLTV(k), and MLPoLTI(k) predictors. This indicates that the benefit of sparsity extends beyond comparison with the specific dense isotropic Gaussian targets used in LeWM to more general dense representations. Moreover, LpWM outperforms VICReg even with the high-capacity Deep-AdaLN(k) predictor. We hypothesize that this is because second-order moment constraints in VICReg alone can be insufficient as anti-collapse targets because they leave the target distribution underspecified.

Overall, these results show that the benefit of sparsity depends on predictor capacity relative to dynamics complexity. In the simple Wall environment, even linear predictors are sufficient, leaving little room for sparsity to improve performance. In the harder PushT environment, sparsity provides substantial gains at intermediate capacities, but these gains diminish with sufficiently expressive predictors. As the underlying dynamics become more complex, the advantages of sparsity may extend to increasingly expressive predictors, potentially making sparse representations a preferable default over dense representations for modeling latent dynamics. We leave a systematic investigation of this hypothesis to future work.

4 Does Sparsity Lead to Interpretable Latent Dynamics?

Section 3 shows that sparse codes can simplify latent prediction. We now ask what structure emerges in these representations. We call a representation mode-factored when its binary support primarily identifies the discrete dynamics regime, while feature magnitudes capture finer within-regime state. We first test whether this structure emerges naturally in Piecewise, where the dynamics mode is known by construction, and then examine whether it extends to OGBench-Cube [Park et al., 2025], where contact provides a natural temporal regime change.

4.1 Discovering Piecewise-Affine Dynamics

Piecewise is a 2D navigation environment partitioned into zones with different force fields, yielding piecewise-affine dynamics (Appendix C.2). Empirically, we observe that success rate reaches 84.7% for LpWM versus 65.3% for LeWM with random goals, while performance is saturated for goals sampled from the set of evaluation episodes for both methods (Table 4).

To test whether the sparse support recovers the ground-truth piecewise structure, we discretize the Piecewise environment into a 20×20 grid, yielding 400 possible agent locations. For each location, we compute the corresponding embedding and binarize it into a support vector in $\{0, 1\}^D$. For two

binary support vectors $\mathbf{x}, \mathbf{y} \in \{0, 1\}^D$, we measure their overlap using the Jaccard index

$$J(\mathbf{x}, \mathbf{y}) = \frac{\sum_{i=1}^D \mathbb{1}_{\mathbf{x}_i=1 \wedge \mathbf{y}_i=1}}{\sum_{i=1}^D \mathbb{1}_{\mathbf{x}_i=1 \vee \mathbf{y}_i=1}}, \quad (5)$$

where $\wedge, \vee$ are logical “and”, “or” symbols. We defer further details and its differentiable relaxation given in Appendix E. We then select a reference cell, marked by $+$ in Figure 2a, and compute its Jaccard similarity with every other cell. The resulting similarity is high within the reference zone and drops sharply across zone boundaries (dashed), a pattern that persists even when the zones contain no visual cues (bottom). This indicates that LpWM recovers the underlying dynamics regime from action-conditioned prediction rather than appearance alone, organizing the mode structure into which features are active.

Linear probes further confirm a factorized code (Figure 2b): the binary support decodes the ground-truth zone at near-perfect accuracy for sparse LpWM, as accurately as the full embedding, while the continuous agent position is decoded better from the magnitudes. Therefore, the support carries the discrete zone mode, and the magnitudes carry the within-zone agent’s state.

Finally, this structure also accompanies strong long-horizon planning (Figure 2c): sparse LpWM outperforms dense LeWM at every horizon H , with the gap widening as H increases.

4.2 From Navigation to Contact-Rich Dynamics

Does the sparse code also separate distinct phases of the dynamics in contact-rich manipulation? On OGBench-Cube, contact provides a natural regime change between free arm motion and arm-cube interaction. As a temporal analogue of the spatial partition in Piecewise, we measure the support instability $1 - J(\mathbf{z}_t, \mathbf{z}_{t+1})$ between consecutive frames and compute its Pearson correlation with effector motion, cube motion, and gripper contact.

Unlike Piecewise, sparsity alone does not determine which temporal factor is encoded by the support. RDMReg constrains only the per-frame marginal, and without an explicit temporal prior the support tends to follow the dominant fast-varying signal. Accordingly, without temporal regularization, support instability behaves largely as a motion detector: it correlates strongly with effector motion, with $r \approx 0.87$, but only weakly with contact, with $r \approx 0.05$ (Figure 4a).

To test whether the support can instead track slower physical regime changes, we additionally consider an optional temporal Jaccard (TJ) loss that encourages support stability across adjacent frames. As the TJ weight increases, support instability becomes less aligned with effector motion and increasingly aligned with cube motion and contact (Figure 4b); for example, the correlation with cube motion increases from $r_{\text{cube}} = 0.21$ to 0.80. The support raster in Figure 4c shows the same effect visually.

These results highlight a limitation of vanilla LpWM: sparsity alone may not ensure that support transitions correspond to semantically meaningful dynamical events, and without temporal structure the support can instead behave largely as a motion detector. However, they also suggest that, with an appropriate temporal prior, sparse support can become an interpretable tracker of physical regime changes such as object interaction and contact. We leave the design of temporal regularization for more complex dynamics to future work.

5 Conclusion

We introduced LpWM, an action-conditioned JEPa regularized with RDMReg to produce non-negative, exactly sparse latents. Motivated by a one-hot linearization result, we showed empirically that sparsity simplifies the action-conditioned latent dynamics: at a fixed, intermediate predictor capacity, a shallow predictor plans over sparse codes where it fails over dense ones on PushT. We further found the sparse code is mode-factored—its support encodes the discrete dynamics regime (94–99% decodable on Piecewise) and, with a temporal-Jaccard prior, tracks contact on OGBench-Cube. Hence sparse representations can reduce the predictor complexity for control while leading to more interpretable dynamics.

Acknowledgement

We thank Amir Bar, Raktim Goswami, Ying Wang, Wancong Zhang, Gaoyue Zhou, and Kefei Zhu for helpful discussions. This work was supported in part by AFOSR under grant FA95502310139 and NSF Award 1922658. This work was also supported through AMI Labs and NYU IT High Performance Computing resources, services, and staff expertise.

References

- Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture, 2023. URL <https://arxiv.org/abs/2301.08243>.
- Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba, Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zhohus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-jepa 2: Self-supervised video models enable understanding, prediction and planning, 2025. URL <https://arxiv.org/abs/2506.09985>.
- Randall Balestriero and Yann LeCun. Lejepa: Provable and scalable self-supervised learning without the heuristics, 2025. URL <https://arxiv.org/abs/2511.08544>.
- Randall Balestriero, Mark Ibrahim, Vlad Sobal, Ari Morcos, Shashank Shekhar, Tom Goldstein, Florian Bordes, Adrien Bardes, Gregoire Mialon, Yuandong Tian, Avi Schwarzschild, Andrew Gordon Wilson, Jonas Geiping, Quentin Garrido, Pierre Fernandez, Amir Bar, Hamed Pirsiavash, Yann LeCun, and Micah Goldblum. A cookbook of self-supervised learning, 2023. URL <https://arxiv.org/abs/2304.12210>.
- Adrien Bardes, Jean Ponce, and Yann LeCun. Vicreg: Variance-invariance-covariance regularization for self-supervised learning, 2022. URL <https://arxiv.org/abs/2105.04906>.
- Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video, 2024. URL <https://arxiv.org/abs/2404.08471>.
- Steven L Brunton, Joshua L Proctor, and J Nathan Kutz. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the national academy of sciences*, 113(15):3932–3937, 2016.
- Deep Chakraborty, Yann LeCun, Tim G. J. Rudner, and Erik Learned-Miller. Improving pre-trained self-supervised embeddings through effective entropy maximization, 2025. URL <https://arxiv.org/abs/2411.15931>.
- Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations, 2020. URL <https://arxiv.org/abs/2002.05709>.
- Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021. URL <https://arxiv.org/abs/2010.11929>.
- Aleksandr Ermolov, Aliaksandr Siarohin, Enver Sangineto, and Nicu Sebe. Whitening for self-supervised representation learning, 2021. URL <https://arxiv.org/abs/2007.06346>.
- Raktim Gautam Goswami, Prashanth Krishnamurthy, and Farshad Khorrami. Data-driven deep learning based feedback linearization of systems with unknown dynamics. In *2023 American Control Conference (ACC)*, pages 66–71. IEEE, 2023.

- Jean-Bastien Grill, Florian Strub, Florent Althé, Corentin Tallec, Pierre H. Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Daniel Guo, Mohammad Gheshlaghi Azar, Bilal Piot, Koray Kavukcuoglu, Rémi Munos, and Michal Valko. Bootstrap your own latent: A new approach to self-supervised learning, 2020. URL <https://arxiv.org/abs/2006.07733>.
- Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models, 2022. URL <https://arxiv.org/abs/2010.02193>.
- Olivier J Henaff, Robbe LT Goris, and Eero P Simoncelli. Perceptual straightening of natural videos. *Nature neuroscience*, 22(6):984–991, 2019.
- Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus), 2023. URL <https://arxiv.org/abs/1606.08415>.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021. URL <https://arxiv.org/abs/2106.09685>.
- Jiaming Hu, Yan Zheng, and Tian Wang. Scale: State-calibrated latent embeddings for jepa planning in the right geometry, 2026. URL <https://arxiv.org/abs/2608.16287>.
- Arnav Kumar Jain, Shivakanth Sujit, Shruti Joshi, Vincent Michalski, Danijar Hafner, and Samira Ebrahimi-Kahou. Learning robust dynamics through variational sparse gating, 2022. URL <https://arxiv.org/abs/2210.11698>.
- Li Jing, Pascal Vincent, Yann LeCun, and Yuandong Tian. Understanding dimensional collapse in contrastive self-supervised learning. *arXiv preprint arXiv:2110.09348*, 2021.
- Rudolf Emil Kalman et al. Contributions to the theory of optimal control. *Bol. soc. mat. mexicana*, 5(2):102–119, 1960.
- Milan Korda and Igor Mezić. Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control. *Automatica*, 93:149–160, 2018.
- Yilun Kuang, Yash Dagade, Deep Chakraborty, Erik Learned-Miller, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Radial-vcreg: More informative representation learning through radial gaussianization, 2026a. URL <https://arxiv.org/abs/2602.14272>.
- Yilun Kuang, Yash Dagade, Tim G. J. Rudner, Randall Balestriero, and Yann LeCun. Rectified lpjepa: Joint-embedding predictive architectures with sparse and maximum-entropy representations, 2026b. URL <https://arxiv.org/abs/2602.01456>.
- Yann LeCun et al. A path towards autonomous machine intelligence version 0.9. 2, 2022-06-27. *Open Review*, 62(1):1–62, 2022.
- Timothée Lesort, Natalia Díaz-Rodríguez, Jean-François Goudou, and David Filliat. State representation learning for control: An overview. *Neural Networks*, 108:379–392, December 2018. ISSN 0893-6080. doi: 10.1016/j.neunet.2018.07.006. URL <http://dx.doi.org/10.1016/j.neunet.2018.07.006>.
- Lucas Maes, Quentin Le Lidec, Luiz Facury, Nassim Massaudi, Ayush Chaurasia, Francesco Capuano, Richard Gao, Taj Gillin, Dan Haramati, Damien Scieur, Yann LeCun, and Randall Balestriero. stable-worldmodel: A platform for reproducible world modeling research and evaluation, 2026a. URL <https://arxiv.org/abs/2605.21800>.
- Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels, 2026b. URL <https://arxiv.org/abs/2603.19312>.
- Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. Ogbench: Benchmarking offline goal-conditioned rl, 2025. URL <https://arxiv.org/abs/2410.20092>.
- William Peebles and Saining Xie. Scalable diffusion models with transformers, 2023. URL <https://arxiv.org/abs/2212.09748>.

- Giordano Pola, Antoine Girard, and Paulo Tabuada. Approximately bisimilar symbolic models for nonlinear control systems. *Automatica*, 44(10):2508–2516, 2008.
- Joshua L Proctor, Steven L Brunton, and J Nathan Kutz. Dynamic mode decomposition with control. *SIAM Journal on Applied Dynamical Systems*, 15(1):142–161, 2016.
- Reuven Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology and computing in applied probability*, 1(2):127–190, 1999.
- Mikuláš Ružička. Anwendung mathematisch-statistischer methoden in der geobotanik: Synthetische bearbeitung von aufnahmen. *Biológia, Bratislava*, 13:647–661, 1958.
- Xu Shang, Masih Haseli, Jorge Cortés, and Yang Zheng. On the existence of koopman linear embeddings for controlled nonlinear systems. *arXiv preprint arXiv:2602.14537*, 2026.
- Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestrieri, Tim G. J. Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models, 2025. URL <https://arxiv.org/abs/2502.14819>.
- Basile Terver, Randall Balestrieri, Megi Dervishi, David Fan, Quentin Garrido, Tushar Nagarajan, Koustuv Sinha, Wancong Zhang, Mike Rabbat, Yann LeCun, and Amir Bar. A lightweight library for energy-based joint-embedding predictive architectures, 2026. URL <https://arxiv.org/abs/2602.03604>.
- Yifei Wang, Qi Zhang, Yaoyu Guo, and Yisen Wang. Non-negative contrastive learning, 2024. URL <https://arxiv.org/abs/2403.12459>.
- Ying Wang, Oumayma Bounou, Yann LeCun, and Mengye Ren. Adajepa: An adaptive latent world model, 2026a. URL <https://arxiv.org/abs/2606.32026>.
- Ying Wang, Oumayma Bounou, Gaoyue Zhou, Randall Balestrieri, Tim G. J. Rudner, Yann LeCun, and Mengye Ren. Temporal straightening for latent planning, 2026b. URL <https://arxiv.org/abs/2603.12231>.
- Greg Yang, Edward J. Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. Tensor programs v: Tuning large neural networks via zero-shot hyperparameter transfer, 2022. URL <https://arxiv.org/abs/2203.03466>.
- Greg Yang, James B. Simon, and Jeremy Bernstein. A spectral condition for feature learning, 2024. URL <https://arxiv.org/abs/2310.17813>.
- Thomas Yerxa, Yilun Kuang, Eero Simoncelli, and SueYeon Chung. Learning efficient coding of natural images with maximum manifold capacity representations, 2023. URL <https://arxiv.org/abs/2303.03307>.
- Jure Zbontar, Li Jing, Ishan Misra, Yann LeCun, and Stéphane Deny. Barlow twins: Self-supervised learning via redundancy reduction, 2021. URL <https://arxiv.org/abs/2103.03230>.
- Wancong Zhang, Basile Terver, Artem Zhohus, Soham Chitnis, Harsh Sutaria, Mido Assran, Randall Balestrieri, Amir Bar, Adrien Bardes, Yann LeCun, and Nicolas Ballas. Hierarchical planning with latent world models, 2026. URL <https://arxiv.org/abs/2604.03208>.
- Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. Dino-wm: World models on pre-trained visual features enable zero-shot planning, 2025. URL <https://arxiv.org/abs/2411.04983>.

A Related Work

Latent world models for planning. Joint-Embedding Predictive Architectures (JEPA) predict in a learned latent space rather than reconstructing pixels [Assran et al., 2023, Bardes et al., 2024]. Equipped with action-conditioning, JEPA-style world models can be viewed as latent dynamics model of the environment, and can be used for planning with model predictive control [Assran et al., 2025, Zhou et al., 2025, Sobal et al., 2025, Maes et al., 2026b]. Hierarchical JEPA has also been shown to outperform standard flat JEPA models with improved planning performances [Zhang et al., 2026].

Table 3: **Sparsity of the LpWM encoder representations across predictor types and latent dimensions.** Each entry is the fraction of non-zero coordinates $\mathbb{E}[\|\mathbf{z}\|_0]/D$ in the sparse code for the target distribution with $\{\mu = 0, \sigma = \sqrt{1/2}, p = 1\}$, measured on the validation set at the tuned-best cell for the LpWM run paired with each predictor at latent dimension D . Dense LeWM has active fraction 1.0 by construction. The learned code stays $\sim 30\text{--}65\%$ active regardless of predictor or dimension, confirming the codes underlying the planning comparison are genuinely sparse.

D	Deep-AdaLN(k)	Shallow-AdaLN(k)	MLP $\circ$ LTV(k)	MLP $\circ$ LTI(k)	LTI(k)	LTI(1)
384	0.34	0.33	0.37	0.28	0.41	0.63
768	0.44	0.37	0.54	0.53	0.41	0.63
1536	0.49	0.39	0.49	0.54	0.41	0.60
2048	0.42	0.46	0.51	0.46	0.43	0.61
4096	0.48	0.49	0.47	0.43	0.44	0.56

Feature Regularization for Latent World Models. JEPA-style world models plan in the latent space, and hence proper regularizations of the embedding matters for both anti-collapse and also better planning performances. Preventing collapse without negative examples [Chen et al., 2020] is the central design problem of non-contrastive SSL [Balestriero et al., 2023]. The stop-gradient and EMA techniques [Grill et al., 2020] have been used in DINO-WM [Zhou et al., 2025], V-JEPA 2 AC [Assran et al., 2025] and related work. More recently, information maximization based techniques have also been proposed, which aims at reducing feature redundancies [Zbontar et al., 2021, Ermolov et al., 2021, Yerxa et al., 2023] and maximizing feature entropy through distribution-matching regularizations [Chakraborty et al., 2025, Balestriero and LeCun, 2025, Kuang et al., 2026a,b].

Beyond collapse preventions, other techniques have been proposed to improve the latent space for planning and control. Wang et al. [2026b] show that adding a temporal straightening loss [Henaff et al., 2019] reduces the curvature of the embeddings across time and hence enables better planning. SCALE [Hu et al., 2026] aligns latent distances with task-relevant state differences to make the planner-facing geometry more informative. Wang et al. [2026a] also considers adapting the final layer features online to improve planning performance and achieves better out-of distribution generalization. It’s also helpful to use inverse-dynamics modeling [Lesort et al., 2018], temporal smoothness [Sobal et al., 2025], which can lead to better downstream planning performances [Terver et al., 2026].

Sparse and Discrete Representations in World Models. Discrete latent representations have previously been explored in world models. DreamerV2 [Hafner et al., 2022] replaces Gaussian stochastic states with a product of categorical variables, which can equivalently be viewed as a structured sparse binary representation, and finds that categorical latents substantially improve performance over Gaussian ones. The authors hypothesize that sparsity may contribute to this improvement and that categorical states may better capture non-smooth transitions. Relatedly, Variational Sparse Gating [Jain et al., 2022] introduces sparsity into latent dynamics by sparsely updating recurrent-state dimensions through stochastic binary gates. In contrast, we study sparsity directly as a geometry for continuous latent representations: LpWM learns distributed, non-negative sparse codes whose support is not fixed by a categorical grouping. Our focus is also different in that we ask whether sparsity reduces the complexity of the action-conditioned transition model itself, and whether the learned support recovers discrete dynamical regimes while the nonzero magnitudes retain continuous within-regime state.

Linearizing Nonlinear Dynamics. Koopman operator theory studies nonlinear autonomous systems through lifted observables with linear evolution, with extensions to controlled systems enabling linear prediction and MPC [Proctor et al., 2016, Korda and Mezić, 2018]. Recent work further characterizes the conditions for which exact finite-dimensional Koopman embeddings exist for controlled nonlinear systems [Shang et al., 2026]. Related approaches include SINDy, which identifies sparse nonlinear governing equations from candidate function libraries [Brunton et al., 2016]. Data-driven feedback linearization also learns state and input transformations of yielding linear controlled dynamics [Goswami et al., 2023].

Table 4: **Success Rate (%) over 50 Test Samples in the Piecewise Environments with Varying Numbers of Zones.** Piecewise ($n \times n$) denotes the environment with varying numbers of zones. Goal images are either sampled from the evaluation dataset or randomly sampled from all possible locations of the dot agent. H denotes the horizon and R represents the receding-horizon. Results are shown as mean $\pm$ std over 3 seeds. **Bold** denotes the best performance.

Env	Method	Goals from Evaluation Set	Random Goals	
		$H = R = 5$	$H = 5, R = 1$	$H = R = 5$
Piecewise (2×2)	LpWM (Ours)	99.33$\pm$1.15	84.67$\pm$4.16	59.33 $\pm$ 2.31
	LpWM+TJ (Ours)	99.33$\pm$1.15	82.67 $\pm$ 5.03	68.67$\pm$4.62
	LeWM	99.33$\pm$1.15	65.33 $\pm$ 4.16	36.00 $\pm$ 3.46
Piecewise (3×3)	LpWM (Ours)	100.00$\pm$0.00	58.67 $\pm$ 1.15	56.00 $\pm$ 4.00
	LpWM+TJ (Ours)	100.00$\pm$0.00	60.00$\pm$2.00	58.00$\pm$5.29
	LeWM	100.00$\pm$0.00	50.00 $\pm$ 5.29	47.33 $\pm$ 6.11

B Predictor Designs

B.1 Overall Designs

Table 1 lists the predictors we compare, in order of decreasing complexity. **Deep-AdaLN**(k) are 6- and 1-block Transformers with AdaLN-zero action conditioning [Peebles and Xie, 2023]; **LTI**(k) and **LTI**(1) are linear (time-invariant) state-space maps over k and 1 history lags; and **MLP $\circ$ LTI**(k) places a single nonlinear readout **W** ReLU($\cdot$) on top of the linear core. One caveat is that we use the RepReLU link function in LpWM when using **LTI**(k) and **LTI**(1) predictors, so we don’t have exact linear dynamics models.

We have also designed **MLP $\circ$ LTV**(k), a data-dependent (linear-time-varying) predictor that moves beyond the fixed weight matrices in the linear-time invariant case.

B.2 MLP $\circ$ LTV(k)

MLP $\circ$ LTV(k) retains the MLP $\circ$ LTI form $\hat{\mathbf{z}}_{t+1} = \sigma(\mathbf{W} \text{ReLU}(\sum_{i=0}^{k-1} \mathbf{A}_i(\mathbf{z}_t) \mathbf{z}_{t-i} + \mathbf{B}(\mathbf{z}_t) \mathbf{a}_t))$ but makes the operator matrices state-dependent. Naively, designing a linear map with a matrix-valued output given a vector-valued input will require a $O(D^3)$ tensor for the feature dimension D .

Instead, we consider a structured matrix parameterization, where the state emits only a small gate that modulates a fixed low-rank correction:

$$\mathbf{A}_i(\mathbf{z}_t) = \mathbf{A}_i + \mathbf{U}_i \text{diag}(\mathbf{g}_i(\mathbf{z}_t)) \mathbf{V}_i^\top, \quad \mathbf{g}(\mathbf{z}_t) = \text{sigmoid}(\mathbf{G} \mathbf{z}_t) \in [0, 1]^r, \quad (6)$$

where $\mathbf{A}_i$ is a fixed base operator, $\mathbf{V}_i^\top: \mathbb{R}^D \rightarrow \mathbb{R}^r$ and $\mathbf{U}_i: \mathbb{R}^r \rightarrow \mathbb{R}^D$ are learned low-rank factors, and the gate $\mathbf{g}(\mathbf{z}_t)$ —a single projection $\mathbf{G}: \mathbb{R}^D \rightarrow \mathbb{R}^{(k+1)r}$ shared across the k lags and the action term—selects among r fixed low-rank “modes” ($r=16$ in our experiments). The action operator $\mathbf{B}(\mathbf{z}_t)$ is gated identically. This costs $O(Dr)$ with no D^3 term. Crucially, only $\mathbf{z}_t$ drives the gate, so the action $\mathbf{a}_t$ still enters affinely: the dynamics core stays control-affine up to the shared nonlinear readout.

We zero-initialize the up-projections $\mathbf{U}_i$ (LoRA-style [Hu et al., 2021]) as an override after μP initialization, so at initialization every correction vanishes and MLP $\circ$ LTV(k) coincides exactly with MLP $\circ$ LTI(k). It is therefore a strict superset that recovers the fixed-operator model unless the data reward state-dependence.

C Environment

C.1 TwoRoom, PushT, and OGBench Cube

We use the saved dataset in Zhou et al. [2025] for TwoRoom, PushT, and in Maes et al. [2026a] for OGBench Cube. During planning, we fix the goal location to be 25 steps away from the starting location and select episodes from the evaluation set. The action block size is fixed to be 5.

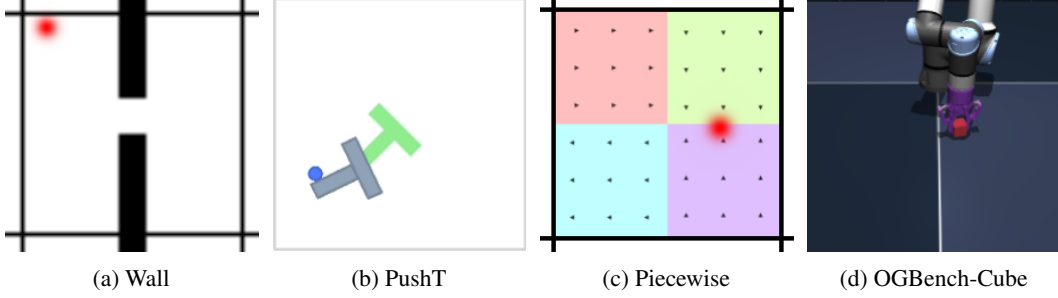


Figure 5: **Environments.** (a) Wall: 2D navigation through a doorway (red agent). (b) PushT: push a T-shaped block to a target pose (green). (c) Piecewise: an open room partitioned into force-field zones (colored) with a known ground-truth segmentation. (d) OGBench-Cube: a robot arm manipulating a cube.

C.2 Piecewise

Piecewise. To measure what the sparse code encodes about the dynamics, we use a synthetic, analytically-known environment called Piecewise [Maes et al., 2026b]. An open 224×224 room (no walls) is divided into a $g \times g$ grid of zones. Each zone i applies a fixed additive bias to motion, $\mathbf{p}_{t+1} = \mathbf{p}_t + \mathbf{a}_t \cdot \text{speed} + \mathbf{b}[\text{zone}(\mathbf{p}_t)]$, with radial biases $\mathbf{b}_i = \text{bias_scale} \cdot (\cos \theta_i, \sin \theta_i)$, $\theta_i = 2\pi i/g^2$. The dynamics are thus locally linear but globally piecewise-affine, with a known ground-truth zone segmentation and an exact expert policy (it inverts the motion equation). This is a controlled environment where the latent mode is known, so we can test directly whether the sparse support recovers it. We use $g \in \{2, 3\}$ (i.e. 4 and 9 zones) in our experiments.

C.3 Visualization of Environments

See Figure 5 for example snapshots of the environments. Note that there is another version of the Piecewise environment without the background color rendering as we have shown in Figure 2a.

D Experimental Details

Our experiments comprise two complementary studies implemented in separate experimental pipelines. Section 3 systematically varies predictor complexity and latent dimension to test whether sparsity reduces the complexity of the learned dynamics in Wall and PushT environments. The interpretability study in Section 4 instead fixes the model configuration and probes the structure of the learned sparse support in the Piecewise and OGBench-Cube environments. The two pipelines instantiate the same underlying LpWM formulation but differ in architecture and optimization details, summarized in Table 5. All comparisons and ablations are performed within the corresponding experimental pipeline.

In both pipelines, we follow the settings in Section 2.1 and Section 2.2 for the loss, architecture, and link function designs. The model is an action-conditioned JEPA trained end-to-end. The encoder and predictor outputs pass through a link function σ , and per-timestep latent marginals are matched by 2-Wasserstein distance to a target distribution obtained by applying the same link to a Generalized Gaussian with shape parameter p and location or mean-shift parameter μ . Dense LeWM uses $\sigma = \text{identity}$ with an isotropic Gaussian target ($p = 2$), whereas sparse LpWM uses $\sigma = (\text{Rep})\text{ReLU}$ with a Rectified Laplace target ($p = 1$), yielding sparse representations.

Experimental Details for Section 3. We follow the settings in DINO-WM [Zhou et al., 2025] and set the context length (k) for the Wall environment to be $k = 1$. PushT uses $k = 3$ because velocity is not recoverable from a single observation. For each predictor type and latent feature dimension configuration, we sweep the RDMReg weight λ_{RDMReg} and the μP base learning rate ℓ_{muP} (Appendix H) and report the best-performing cell in the corresponding grid. We also swept the hyperparameter for LeWM and its VICReg variants in Appendix H to make sure we obtain strong baseline performances.

Table 5: **Experimental configurations for Section 3 and Section 4.** We list out the experimental pipelines in the following table.

	Section 3		Section 4	
	Wall	PushT	Piecewise	OGBench-Cube
Encoder	from-scratch ViT, 12 layers, width 384		from-scratch ViT-Tiny, width 192	
Predictor	See Table 1		Deep-AdaLN(k)	
Dimension (D)	{384, 768, 1536, 2048, 4096}		192	
History k	1	3	3	3
Frameskip	5	5	5	5
Batch size	128	64	128	128
Epochs	20	2	10	10
Optimizer	Adam, μ P (with base width 384)		AdamW	
Base LR	See Appendix H		5×10^{-5}	
Grad. clip	0.0		1.0	
Reg. weight λ	See Appendix H		25	10
Number of projections	8192		1024	
Parameter μ	{0, -1, -2}		0	0
Temporal Jaccard	off	off	{0.0, 0.01}	{0.0, 0.005, 0.01, 0.05, 0.1}

Experimental Details for Section 4. We follow the settings in stable-worldmodel [Maes et al., 2026a]. For both the Piecewise and OGBench-Cube environments, we use a from-scratch ViT-Tiny encoder (width 192, CLS token), latent dimension $D = 192$, and a depth-6 Deep-AdaLN(k) predictor with history $k = 3$. Models are trained for 10 epochs using AdamW with learning rate 5×10^{-5} , weight decay 10^{-3} , a linear-warmup cosine schedule with warmup over approximately 1% of the optimization steps, batch size 128, bf16 precision, and gradient clipping at 1.0. These experiments do not use μ P since the feature dimension is relatively small, i.e. $D = 192$.

Planning and Evaluation. All experiments use the cross-entropy method (CEM) to optimize action sequences in latent space by minimizing the MSE between predicted latent rollouts and the encoded goal representation. Both experimental pipelines use 300 candidate action sequences per CEM iteration, retain the top 30 best-performing action sequences, and run CEM for 30 iterations with an initial variance scale of 1.0 and planning horizon $H = 5$. With a frameskip of 5, the goal observation is therefore 25 raw environment steps ahead.

In Section 3, we report both open-loop and closed-loop planning success rates. Open-loop evaluation executes a single CEM plan without replanning and therefore provides a more direct measure of accumulated model-prediction quality. Closed-loop evaluation uses receding-horizon MPC, replanning after each block of 5 executed actions, with at most 10 replanning iterations. Wall and PushT are evaluated over 50 trajectories whose goals are drawn from the evaluation dataset.

In Section 4, we additionally evaluate Piecewise using randomly sampled goal images, since performance for both LeWM and LpWM saturates when goals are sampled from the evaluation set. In Table 4, we also consider a receding horizon of 1. For OGBench-Cube, we follow the evaluation protocol of Maes et al. [2026b], where goal observations are sampled only 25 raw environment steps ahead. This short horizon can make the planning task relatively easy, potentially leading to performance saturation and limiting the discriminative power of the benchmark. We leave evaluation under more challenging longer-horizon and larger-frameskip settings to future work.

E Jaccard Index

E.1 Hard and Soft Jaccard index

The standard Jaccard index between two sets A and B is $\text{Jac}(A, B) = |A \cap B| / |A \cup B|$. For two binary vectors $\mathbf{x}, \mathbf{y} \in \{0, 1\}^D$ this is $J(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^D \mathbb{1}_{\mathbf{x}_i=1 \wedge \mathbf{y}_i=1} / \sum_{i=1}^D \mathbb{1}_{\mathbf{x}_i=1 \vee \mathbf{y}_i=1}$, where $\wedge, \vee$ are logical “and”, “or” symbols. Because the encoder’s output is non-negative but not binary, directly optimizing this set Jaccard is non-differentiable. We therefore use the soft relaxation [Ružička, 1958],

for $\mathbf{x}, \mathbf{y} \in \mathbb{R}_{\geq 0}^D$,

$$J_S(\mathbf{x}, \mathbf{y}) = \frac{\sum_{i=1}^D \min(\mathbf{x}_i, \mathbf{y}_i)}{\sum_{i=1}^D \max(\mathbf{x}_i, \mathbf{y}_i) + \epsilon}, \quad (7)$$

where $\min(\cdot, \cdot)$ and $\max(\cdot, \cdot)$ are smooth proxies for $\wedge$ and $\vee$; a higher J_S means more similar supports, and J_S recovers the exact set Jaccard when its arguments are binary.

E.2 Temporal Jaccard Loss

Enforcing the feature distribution to be sparse independently across time steps via RDMReg imposes no constraints on the desired temporal transition. Therefore we also consider the optional temporal Jaccard loss (TJ) as a slowness prior on the binary support vector. The temporal Jaccard loss penalizes rapid support transitions between consecutive frames and hence encourages the support to be smoothly varying across time while not interfering with the global sparsity constraints brought by RDMReg.

Temporal Jaccard loss. Given encoder embeddings $\mathbf{Z} \in \mathbb{R}_{\geq 0}^{B \times T \times D}$, where B is batch size, T is the temporal dimension, and D is the feature dimension, the temporal jaccard loss is the mean support instability over all consecutive frame pairs:

$$\mathcal{L}_{\text{TJ}} = \frac{1}{B(T-1)} \sum_{b=1}^B \sum_{t=1}^{T-1} \left(1 - J_S(\mathbf{Z}[b, t, :], \mathbf{Z}[b, t+1, :])\right) \quad (8)$$

where $\mathbf{Z}[b, t, :] \in \mathbb{R}^D$ denotes the slice of the embedding tensor at batch index b and time index t . Minimizing $\mathcal{L}_{\text{TJ}}$ encourages the support between embeddings of consecutive frames to be smoothly varying, and hence reduces the chance of drastic support changes between temporally adjacent frames.

F Maximal-Update Parameterization (μP) for JEPA

Our predictor-complexity experiments vary the latent dimensions over more than an order of magnitude, from $D = 384$ to $D = 4096$. A naive parameterization uses one learning rate for all layers in the neural network as D varies, so performance differences across widths can be confounded by width-specific optimization effects rather than properties of the learned representation. We therefore use maximal-update parameterization (μP) [Yang et al., 2022] to preserve feature-learning dynamics across width and enable learning-rate transfer.

For a dense linear map $\mathbf{h} = \mathbf{W}\mathbf{x}$ with $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, μP chooses initialization and learning-rate scalings such that both the hidden activations and their training-induced updates remain of order $\Theta(\sqrt{d_{\text{out}}})$ as width changes. Equivalently, the weight and update operators remain at the appropriate spectral scale for stable feature learning [Yang et al., 2024]. For Adam-type optimizers, this gives an effective learning-rate scaling proportional to the inverse fan-in,

$$\eta_{\mathbf{W}} \propto \frac{1}{d_{\text{in}}}. \quad (9)$$

We apply this rule to all width-dependent dense maps in the JEPA encoder and predictor. Let D_0 denote the reference latent width at which a base learning rate η_{base} is selected. In our experiment, we fix $D_0 = 384$. For parameter matrices whose fan-in scales linearly with the latent dimension D , we use the following scaling equation:

$$\eta_{\mathbf{W}}(D) = \eta_{\text{base}} \frac{D_0}{D}. \quad (10)$$

Width-independent parameter groups retain their base learning rate. Initialization follows the corresponding μP fan-in scaling. For all other layers or components like layer norm or bias, we follow the design recipe in Appendix B.1 in Yang et al. [2022].

Given that μP has not been demonstrated in JEPA architectures before for hyperparameter transfer across model width to our best knowledge, we don't intend to use μP to reduce the hyperparameter tuning efforts. Instead, we use μP here simply to ensure stable feature learning, i.e. maintaining a stable spectral norm of the weight matrices to be around $\|\mathbf{W}\|_* = \Theta(\sqrt{d_{\text{out}}/d_{\text{in}}})$.

In Appendix H, we show the μP parameterization of learning rate can transfer across width most of the time, although there are occasional exceptions. We intend to investigate this more in future work.

G Proofs of Linearization

G.1 Proof of Proposition 1

Proof. Since $\mathcal{X}$ is a compact subset of $\mathbb{R}^d$, the open cover of $\mathcal{X}$

$$\mathcal{X} \subseteq \bigcup_{\mathbf{c} \in \mathcal{X}} B_\varepsilon(\mathbf{c}) \quad (11)$$

contains a finite subcover. In other words, for every $\varepsilon > 0$, there exists $C_\varepsilon = \{\mathbf{c}_1, \dots, \mathbf{c}_N\} \subset \mathcal{X}$ such that

$$\mathcal{X} \subseteq \bigcup_{\mathbf{c} \in C_\varepsilon} B_\varepsilon(\mathbf{c}) \quad (12)$$

where $B_\varepsilon(\mathbf{c}) := \{\mathbf{x} \in \mathcal{X} : \|\mathbf{x} - \mathbf{c}\| < \varepsilon\}$. Now consider a deterministic quantization map $Q_\varepsilon : \mathcal{X} \rightarrow \{1, \dots, N\}$ such that $\|\mathbf{x} - \mathbf{c}_{Q_\varepsilon(\mathbf{x})}\| \leq \varepsilon$ for all $\mathbf{x} \in \mathcal{X}$.

Let's define the encoder as $E_\varepsilon(\mathbf{x}) = \mathbf{e}_{Q_\varepsilon(\mathbf{x})}$, where $\mathbf{e}_i$ is the i -th standard basis vector of $\mathbb{R}^N$. Hence $\|E_\varepsilon(\mathbf{x})\|_0 = 1$ for every $\mathbf{x} \in \mathcal{X}$, i.e. $E_\varepsilon(\cdot)$ is a one-hot encoder.

Correspondingly, we also define the decoder by $D_\varepsilon(\mathbf{e}_i) = \mathbf{c}_i$. Now fix any action $\mathbf{a} \in \mathcal{A}$. For each representative $\mathbf{c}_i$, define $\tau_{\mathbf{a}}(i) := Q_\varepsilon(f(\mathbf{c}_i, \mathbf{a}))$ as the transition dynamics map. Then the transition matrix $\mathbf{P}_\varepsilon(\mathbf{a})$ can be defined column-wise by $\mathbf{P}_\varepsilon(\mathbf{a})\mathbf{e}_i = \mathbf{e}_{\tau_{\mathbf{a}}(i)}$.

Thus every column of $\mathbf{P}_\varepsilon(\mathbf{a})$ contains exactly one nonzero entry, and

$$\mathbf{z}_{t+1} = \mathbf{P}_\varepsilon(\mathbf{a}_t)\mathbf{z}_t \quad (13)$$

is exactly linear in $\mathbf{z}$. It remains to bound the decoded one-step error. Fix arbitrary $\mathbf{x}_t \in \mathcal{X}$ and $\mathbf{a}_t \in \mathcal{A}$, and let $i := Q_\varepsilon(\mathbf{x}_t)$. Then by the definition of the encoder we have $E_\varepsilon(\mathbf{x}_t) = \mathbf{e}_i$ with $\|\mathbf{x}_t - \mathbf{c}_i\| \leq \varepsilon$. We can rollout the latent dynamics to obtain

$$\mathbf{P}_\varepsilon(\mathbf{a}_t)E_\varepsilon(\mathbf{x}_t) = \mathbf{P}_\varepsilon(\mathbf{a}_t)\mathbf{e}_i = \mathbf{e}_{\tau_{\mathbf{a}_t}(i)} \quad (14)$$

The decoded state is thus

$$\hat{\mathbf{x}}_{t+1} := D_\varepsilon(\mathbf{P}_\varepsilon(\mathbf{a}_t)E_\varepsilon(\mathbf{x}_t)) = \mathbf{c}_{Q_\varepsilon(f(\mathbf{c}_i, \mathbf{a}_t))}. \quad (15)$$

Let the ground-truth next state be

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{a}_t). \quad (16)$$

By the triangle inequality, we have

$$\|\mathbf{x}_{t+1} - \hat{\mathbf{x}}_{t+1}\| = \|f(\mathbf{x}_t, \mathbf{a}_t) - \mathbf{c}_{Q_\varepsilon(f(\mathbf{c}_i, \mathbf{a}_t))}\| \quad (17)$$

$$\leq \|f(\mathbf{x}_t, \mathbf{a}_t) - f(\mathbf{c}_i, \mathbf{a}_t)\| + \|f(\mathbf{c}_i, \mathbf{a}_t) - \mathbf{c}_{Q_\varepsilon(f(\mathbf{c}_i, \mathbf{a}_t))}\|. \quad (18)$$

Since f is Lipschitz continuous in state, we know

$$\|f(\mathbf{x}_t, \mathbf{a}_t) - f(\mathbf{c}_i, \mathbf{a}_t)\| \leq L\|\mathbf{x}_t - \mathbf{c}_i\| \leq L\varepsilon. \quad (19)$$

Now because $f(\mathbf{c}_i, \mathbf{a}_t) \in \mathcal{X}$, the compactness property implies that

$$\|f(\mathbf{c}_i, \mathbf{a}_t) - \mathbf{c}_{Q_\varepsilon(f(\mathbf{c}_i, \mathbf{a}_t))}\| \leq \varepsilon. \quad (20)$$

Therefore the approximation error is upper bounded by

$$\|\mathbf{x}_{t+1} - \hat{\mathbf{x}}_{t+1}\| \leq L\varepsilon + \varepsilon = (L+1)\varepsilon. \quad (21)$$

Since the bound is uniform over $\mathbf{x}_t$ and $\mathbf{a}_t$,

$$\sup_{\mathbf{x}_t \in \mathcal{X}, \mathbf{a}_t \in \mathcal{A}} \|f(\mathbf{x}_t, \mathbf{a}_t) - D_\varepsilon(\mathbf{P}_\varepsilon(\mathbf{a}_t)E_\varepsilon(\mathbf{x}_t))\| \leq (L+1)\varepsilon. \quad (22)$$

Now since the Lipschitz constant L is fixed, we know that $(L+1)\varepsilon \rightarrow 0$ as $\varepsilon \rightarrow 0$. Therefore, we arrive at

$$\lim_{\varepsilon \rightarrow 0} \sup_{\mathbf{x}_t \in \mathcal{X}, \mathbf{a}_t \in \mathcal{A}} \|f(\mathbf{x}_t, \mathbf{a}_t) - D_\varepsilon(\mathbf{P}_\varepsilon(\mathbf{a}_t)E_\varepsilon(\mathbf{x}_t))\| = 0. \quad (23)$$

□

Lemma 1 (Finite-horizon rollout bound). *Under the assumptions of Proposition 1, let $\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{a}_t)$ for $t = 0, \dots, H-1$ be the ground-truth trajectory under an arbitrary action sequence $\mathbf{a}_0, \dots, \mathbf{a}_{H-1} \in \mathcal{A}$. Initialize $\mathbf{z}_0 = E_\varepsilon(\mathbf{x}_0)$ and recursively define the latent transition as $\mathbf{z}_{t+1} = \mathbf{P}_\varepsilon(\mathbf{a}_t)\mathbf{z}_t$. Let $\hat{\mathbf{x}}_t = D_\varepsilon(\mathbf{z}_t)$ be the decoded state. Then*

$$\|\mathbf{x}_H - \hat{\mathbf{x}}_H\| \leq \varepsilon \sum_{k=0}^H L^k. \quad (24)$$

Proof. Define the decoded quantization map as $q_\varepsilon(\mathbf{x}) := D_\varepsilon(E_\varepsilon(\mathbf{x})) = \mathbf{c}_{Q_\varepsilon(\mathbf{x})}$. By construction, $\|\mathbf{x} - q_\varepsilon(\mathbf{x})\| \leq \varepsilon$ for all $\mathbf{x} \in \mathcal{X}$. Since every latent state $\mathbf{z}_t$ is one-hot, we know that if $\mathbf{z}_t = \mathbf{e}_i$, then $\hat{\mathbf{x}}_t = D_\varepsilon(\mathbf{z}_t) = \mathbf{c}_i$. Since the latent dynamics give

$$\mathbf{z}_{t+1} = \mathbf{P}_\varepsilon(\mathbf{a}_t)\mathbf{e}_i = \mathbf{e}_{Q_\varepsilon(f(\mathbf{c}_i, \mathbf{a}_t))}. \quad (25)$$

We know that

$$\hat{\mathbf{x}}_{t+1} = D_\varepsilon(\mathbf{z}_{t+1}) = q_\varepsilon(f(\hat{\mathbf{x}}_t, \mathbf{a}_t)). \quad (26)$$

Define the scalar rollout error as $e_t := \|\mathbf{x}_t - \hat{\mathbf{x}}_t\|$. Then

$$e_{t+1} = \|f(\mathbf{x}_t, \mathbf{a}_t) - q_\varepsilon(f(\hat{\mathbf{x}}_t, \mathbf{a}_t))\| \quad (27)$$

$$\begin{aligned} &\leq \|f(\mathbf{x}_t, \mathbf{a}_t) - f(\hat{\mathbf{x}}_t, \mathbf{a}_t)\| + \|f(\hat{\mathbf{x}}_t, \mathbf{a}_t) - q_\varepsilon(f(\hat{\mathbf{x}}_t, \mathbf{a}_t))\| \\ &\leq L e_t + \varepsilon. \end{aligned} \quad (28)$$

At $t = 0$, $e_0 = \|\mathbf{x}_0 - \hat{\mathbf{x}}_0\| \leq \varepsilon$. Rolling out H steps give us

$$e_H \leq L^H e_0 + \varepsilon \sum_{k=0}^{H-1} L^k \leq L^H \varepsilon + \varepsilon \sum_{k=0}^{H-1} L^k = \varepsilon \sum_{k=0}^H L^k. \quad (29)$$

Hence we arrive at our conclusion

$$\|\mathbf{x}_H - \hat{\mathbf{x}}_H\| \leq \varepsilon \sum_{k=0}^H L^k. \quad (30)$$

□

G.2 Proof of Corollary 1

Proof. Partition each coordinate interval $[0, 1]$ into n subintervals of equal length $h = \frac{1}{n}$. The resulting partition of $[0, 1]^d$ consists of $n^d = N$ axis-aligned hypercubes, each with side length h .

Let $\mathbf{c}_i$ denote the center of one such hypercube. For any point $\mathbf{x}$ in that cell, each coordinate differs from the corresponding coordinate of $\mathbf{c}_i$ by at most $h/2$. Hence

$$|\mathbf{x}_j - \mathbf{c}_{i,j}| \leq \frac{h}{2}, \quad j = 1, \dots, d. \quad (31)$$

Therefore, using the Euclidean norm,

$$\|\mathbf{x} - \mathbf{c}_i\|^2 = \sum_{j=1}^d |\mathbf{x}_j - \mathbf{c}_{i,j}|^2 \leq \sum_{j=1}^d \left(\frac{h}{2}\right)^2 = d \frac{h^2}{4}. \quad (32)$$

Taking square roots gives $\|\mathbf{x} - \mathbf{c}_i\| \leq \frac{\sqrt{d}}{2} h$. Since $h = 1/n$, we arrive at

$$\varepsilon_N = \frac{\sqrt{d}}{2n} = \frac{\sqrt{d}}{2} N^{-1/d}. \quad (33)$$

Based on Proposition 1, the one-step decoded prediction error is bounded by

$$\sup_{\mathbf{x}, \mathbf{a}} \|f(\mathbf{x}, \mathbf{a}) - D_{\varepsilon_N}(\mathbf{P}_{\varepsilon_N}(\mathbf{a})E_{\varepsilon_N}(\mathbf{x}))\| \leq (L+1)\varepsilon_N. \quad (34)$$

Substituting the expression for ε_N gives

$$\sup_{\mathbf{x}, \mathbf{a}} \|f(\mathbf{x}, \mathbf{a}) - D_{\varepsilon_N}(\mathbf{P}_{\varepsilon_N}(\mathbf{a})E_{\varepsilon_N}(\mathbf{x}))\| \leq (L+1) \frac{\sqrt{d}}{2} N^{-1/d} = \frac{\sqrt{d}}{2} (L+1) N^{-1/d}. \quad (35)$$

Since d and L are fixed constants,

$$\sup_{\mathbf{x}, \mathbf{a}} \|f(\mathbf{x}, \mathbf{a}) - D_{\varepsilon_N}(\mathbf{P}_{\varepsilon_N}(\mathbf{a})E_{\varepsilon_N}(\mathbf{x}))\| = O(N^{-1/d}). \quad (36)$$

Similarly, the finite-horizon rollout bound from Lemma 1 gives

$$\|\mathbf{x}_H - \hat{\mathbf{x}}_H\| \leq \varepsilon_N \sum_{k=0}^H L^k. \quad (37)$$

Substituting the covering radius, we have

$$\|\mathbf{x}_H - \hat{\mathbf{x}}_H\| \leq \frac{\sqrt{d}}{2} N^{-1/d} \sum_{k=0}^H L^k. \quad (38)$$

Since H , d , and L are fixed, it follows that

$$\lim_{N \rightarrow \infty} \|\mathbf{x}_H - \hat{\mathbf{x}}_H\| = 0. \quad (39)$$

□

H Hyperparameter Sweeps

H.1 LpWM and LeWM Sweeps in the PushT Environment

To ensure full reproducibility and transparency, we perform and report hyperparameter sweeps underlying Figure 1b.

We run a grid search over the RDMReg regularization weight $\lambda_{\text{RDMReg}} \in \{0.01, 0.1, 1.0, 10.0\}$ and the muP (or μP) learning rate $\ell_{\text{muP}} \in \{0.05, 0.005, 0.0005, 0.00005\}$ for each fixed feature dimension $d \in \{384, 768, 1536, 2048, 4096\}$, method (LeWM and LpWM with target distribution $\prod_{i=1}^d \text{ReLU}(\text{Laplace}(\mu = 0, \sigma = \sqrt{1/2}))$), and predictor setting in Table 1. To ensure fair comparison and standardized hyperparameter sweeps, we use our sliced Wasserstein distance formulation as the distribution-matching loss for isotropic Gaussian targets instead of the Epps-Pulley loss in SIGReg [Balestrieri and LeCun, 2025]. They are both in fact instantiations of Cramer-Wold theorem, and only differ in one-sample vs. two-sample style distribution matching [Kuang et al., 2026b].

Due to compute constraints, we fix one seed for training but average over 3 seeds during planning evaluations with Cross-Entropy Methods (CEM) [Rubinstein, 1999]. In practice, we execute our sweep sequentially. So there are some predictor types where we have done a very thorough grid search. After we have determined that some hyperparameters are always leading to bad performances (e.g. usually learning rate like 0.05 is too large), we drop them in our subsequent sweeps so the resulting figures for different predictor types usually cover different subset of the full grid.

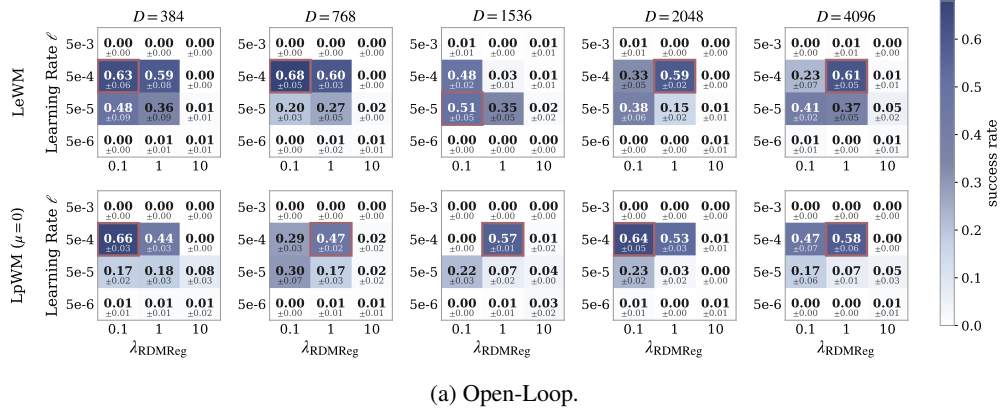
In Figure 9, we observe that $\ell_{\text{muP}} \in \{0.05, 0.005\}$ almost always leads to 0% success rate. Subsequent hyperparameter sweeps shown in Figure 6, Figure 7, Figure 8, Figure 11, and Figure 10 selectively remove portions of the grid to save compute.

H.2 VICReg Sweep in the PushT Environment

We also perform extensive hyperparameter sweeps for JEPa regularized with VICReg. Basically, we take the LeWM setting and replaced the SIGReg regularizer with VICReg. The resulting model doesn't have a name yet, and we abuse the notation and simply call it VICReg here.

In Figure 12, Figure 13, and Figure 14, we show the sweep results for the predictors Deep-AdaLN(k), MLPoLTV(k), and MLPoLTI(k) respectively.

Deep-AdaLN(k) — Open-Loop plan success over $\lambda_{\text{RDMReg}} \times \text{Learning Rate } \ell$ (mean $\pm$ std, 3 seeds)



Deep-AdaLN(k) — Closed-Loop plan success over $\lambda_{\text{RDMReg}} \times \text{Learning Rate } \ell$ (mean $\pm$ std, 3 seeds)

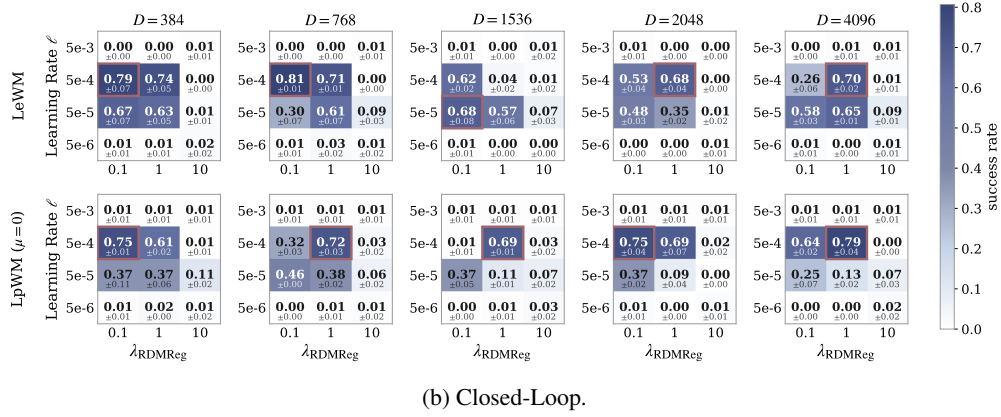
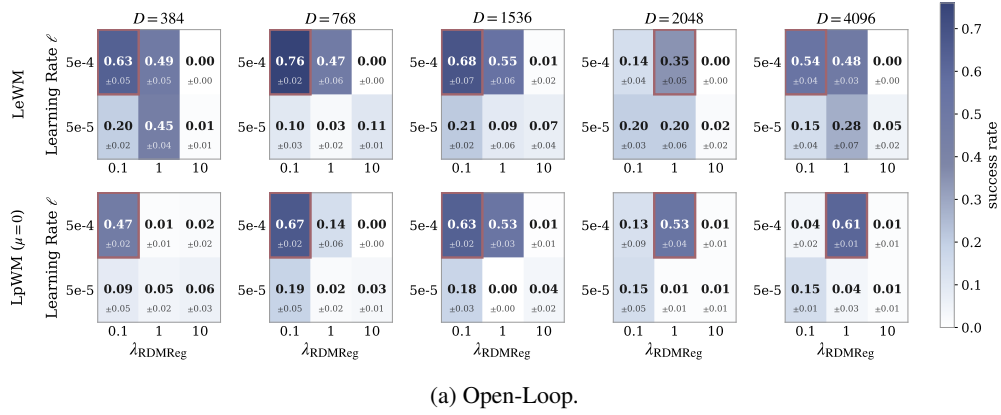


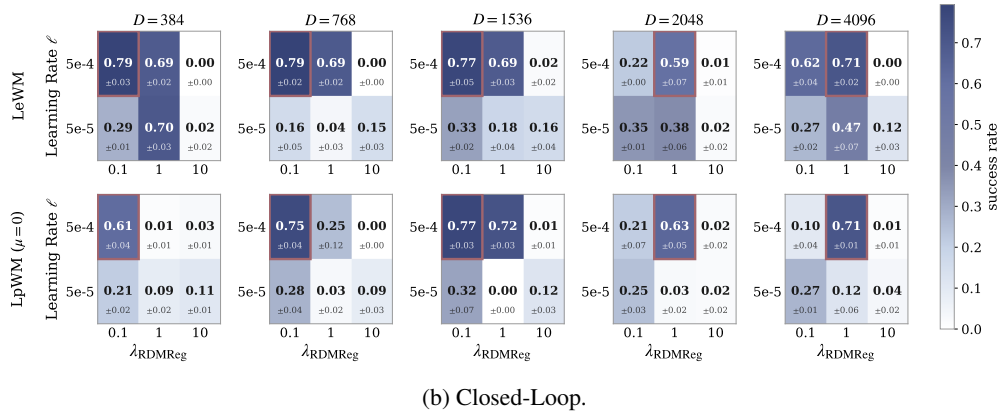
Figure 6: **Deep-AdaLN(k) Predictor.** Hyperparameter Sweeps for the PushT Environment.

Shallow-AdaLN(k) — Open-Loop plan success over $\lambda_{\text{RDMReg}} \times \text{Learning Rate } \ell$ (mean $\pm$ std, 3 seeds)



(a) Open-Loop.

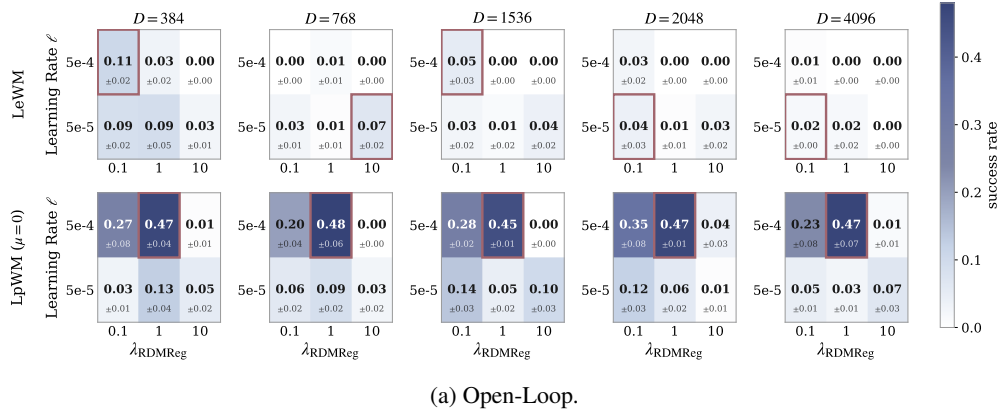
Shallow-AdaLN(k) — Closed-Loop plan success over $\lambda_{\text{RDMReg}} \times \text{Learning Rate } \ell$ (mean $\pm$ std, 3 seeds)



(b) Closed-Loop.

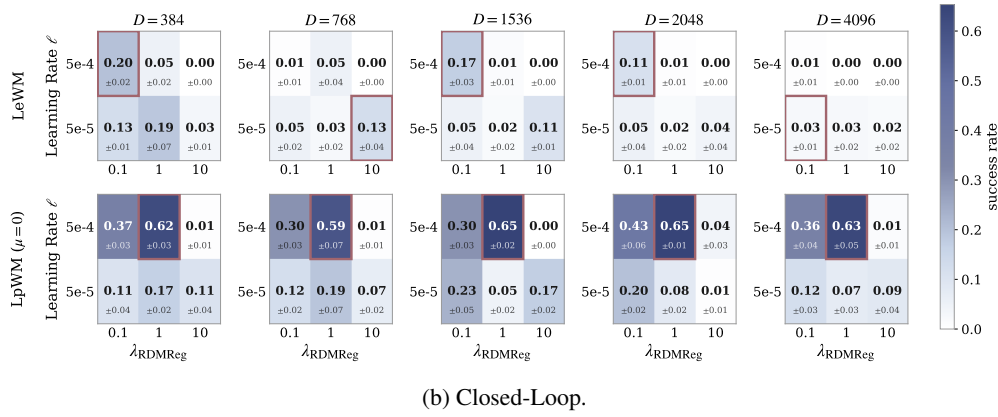
Figure 7: **Shallow-AdaLN(k) Predictor.** Hyperparameter Sweeps for the PushT Environment.

MLP-LTV(k) — Open-Loop plan success over $\lambda_{\text{RDMReg}} \times \text{Learning Rate } \ell$ (mean $\pm$ std, 3 seeds)



(a) Open-Loop.

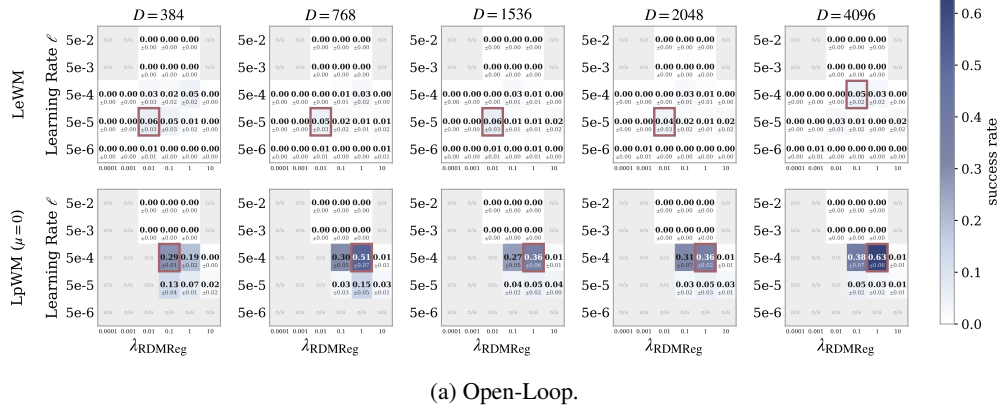
MLP-LTV(k) — Closed-Loop plan success over $\lambda_{\text{RDMReg}} \times \text{Learning Rate } \ell$ (mean $\pm$ std, 3 seeds)



(b) Closed-Loop.

Figure 8: **MLP-LTV(k) Predictor.** Hyperparameter Sweeps for the PushT Environment.

MLP•LTI(k) — Open-Loop plan success over $\lambda_{\text{RDMReg}} \times \text{Learning Rate } \ell$ (mean $\pm$ std, 3 seeds)



MLP•LTI(k) — Closed-Loop plan success over $\lambda_{\text{RDMReg}} \times \text{Learning Rate } \ell$ (mean $\pm$ std, 3 seeds)

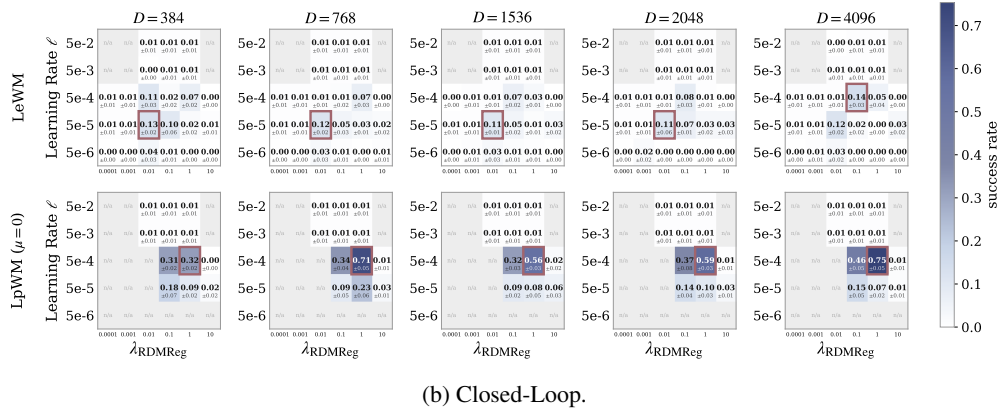


Figure 9: **MLP•LTI(k) Predictor.** Hyperparameter Sweeps for the PushT Environment.

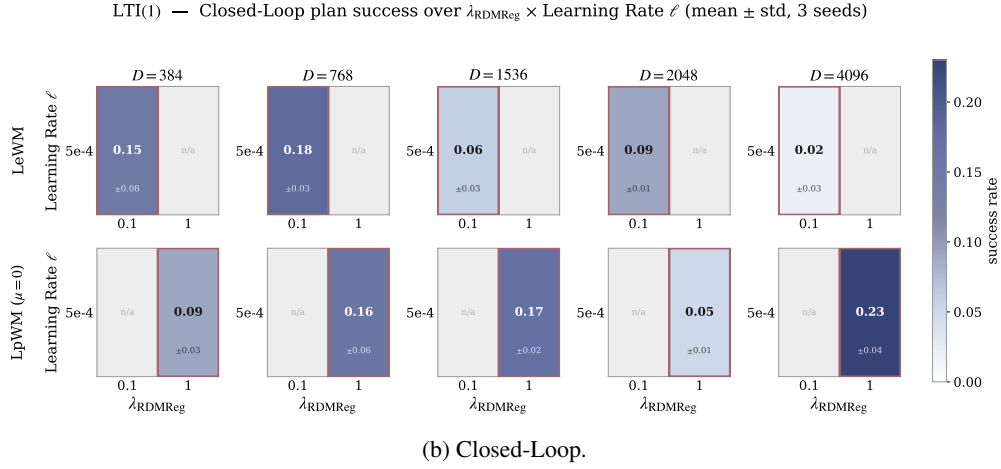
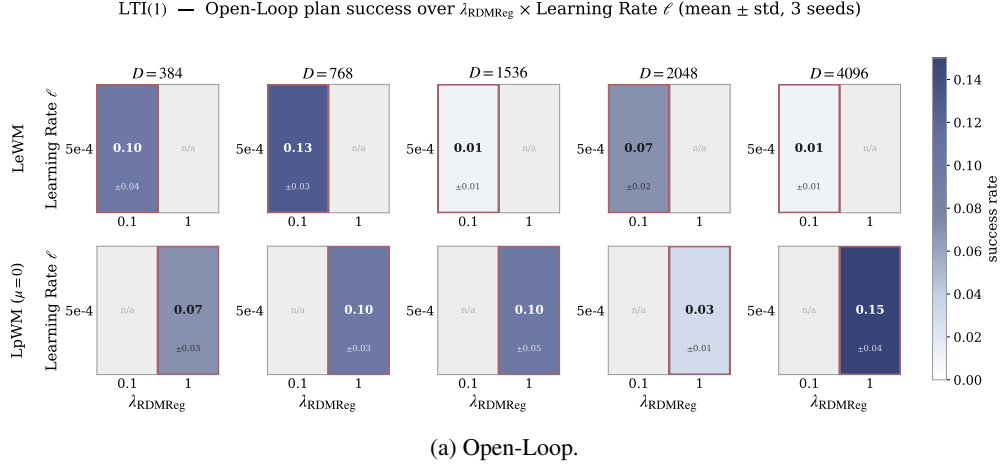
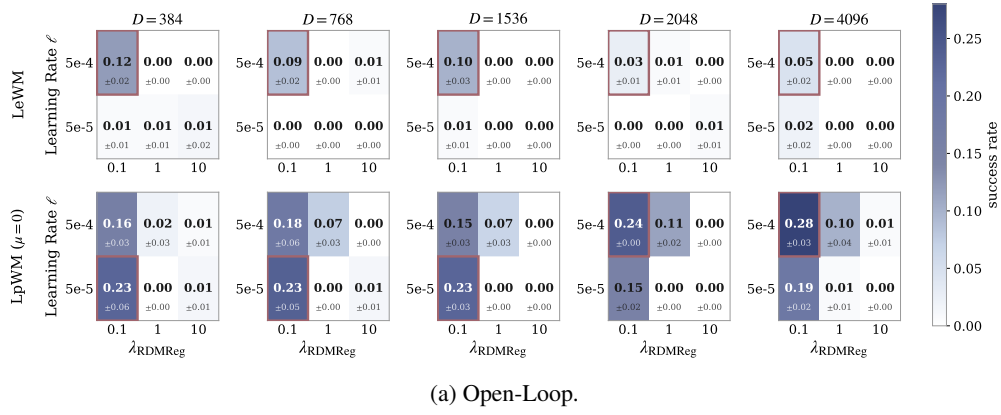


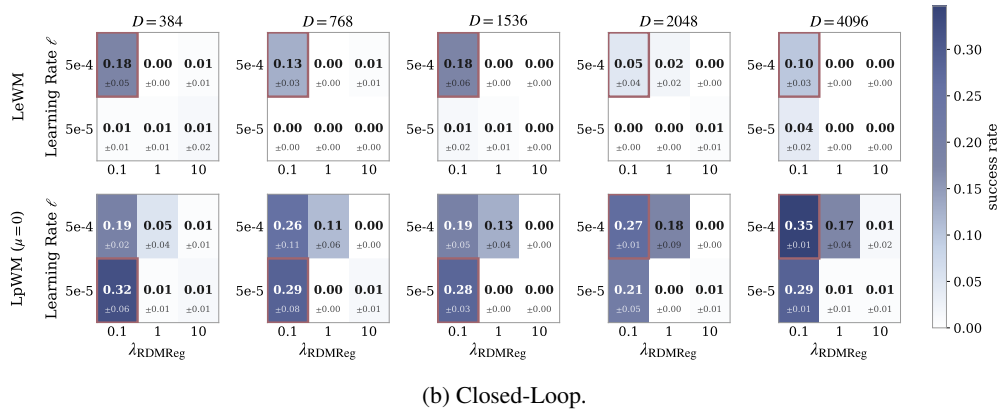
Figure 10: **LTI(1) Predictor.** Hyperparameter Sweeps for the PushT Environment.

LTI(k) — Open-Loop plan success over $\lambda_{\text{RDMReg}} \times \text{Learning Rate } \ell$ (mean $\pm$ std, 3 seeds)



(a) Open-Loop.

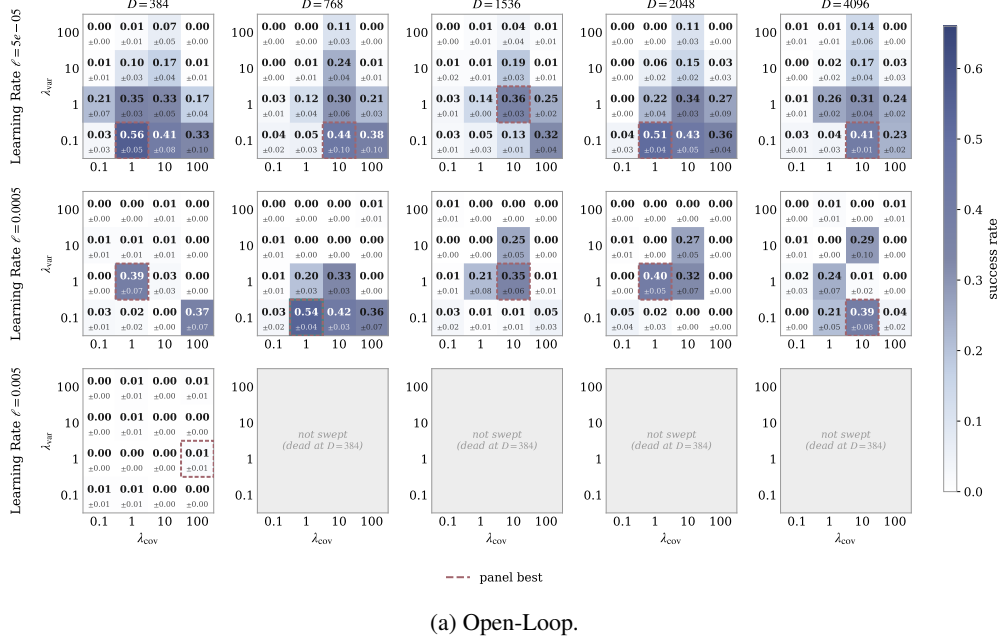
LTI(k) — Closed-Loop plan success over $\lambda_{\text{RDMReg}} \times \text{Learning Rate } \ell$ (mean $\pm$ std, 3 seeds)



(b) Closed-Loop.

Figure 11: LTI(k) Predictor. Hyperparameter Sweeps for the PushT Environment.

Dense-VICReg — Deep-AdaLN(k) — Open-Loop plan success (mean $\pm$ std, 3 seeds)



Dense-VICReg — Deep-AdaLN(k) — Closed-Loop plan success (mean $\pm$ std, 3 seeds)

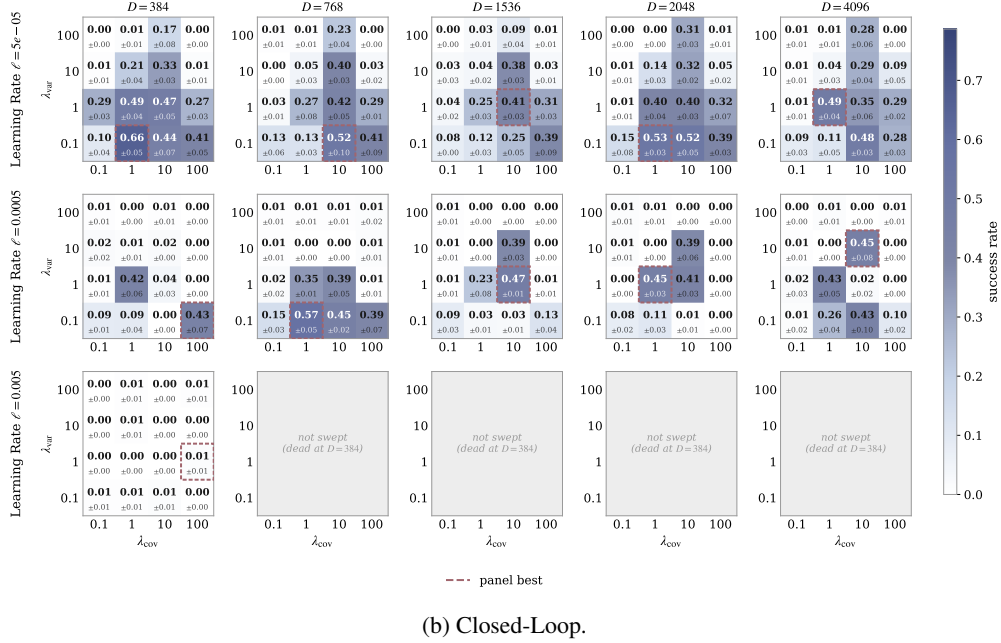
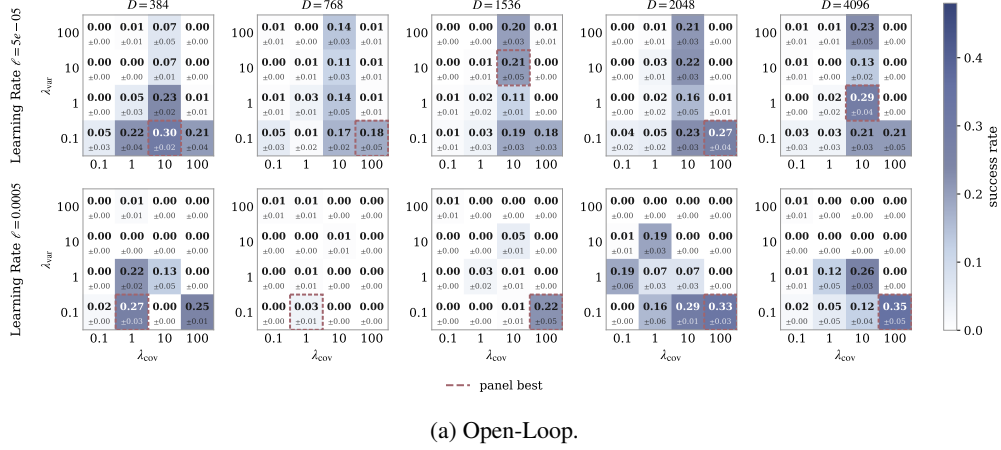


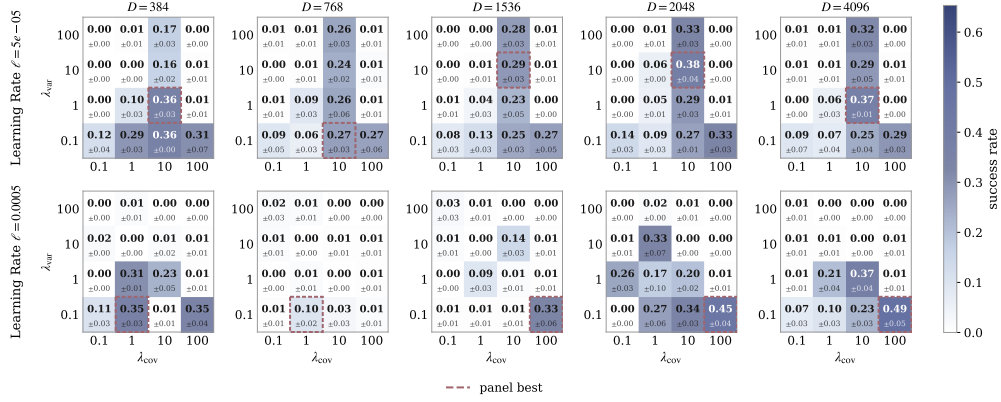
Figure 12: **Deep-AdaLN(k) Predictor.** Hyperparameter Sweeps for VICReg-based JEPa models in the PushT Environment.

Dense-VICReg — MLP•LTV(k) — Open-Loop plan success (mean $\pm$ std, 3 seeds)



(a) Open-Loop.

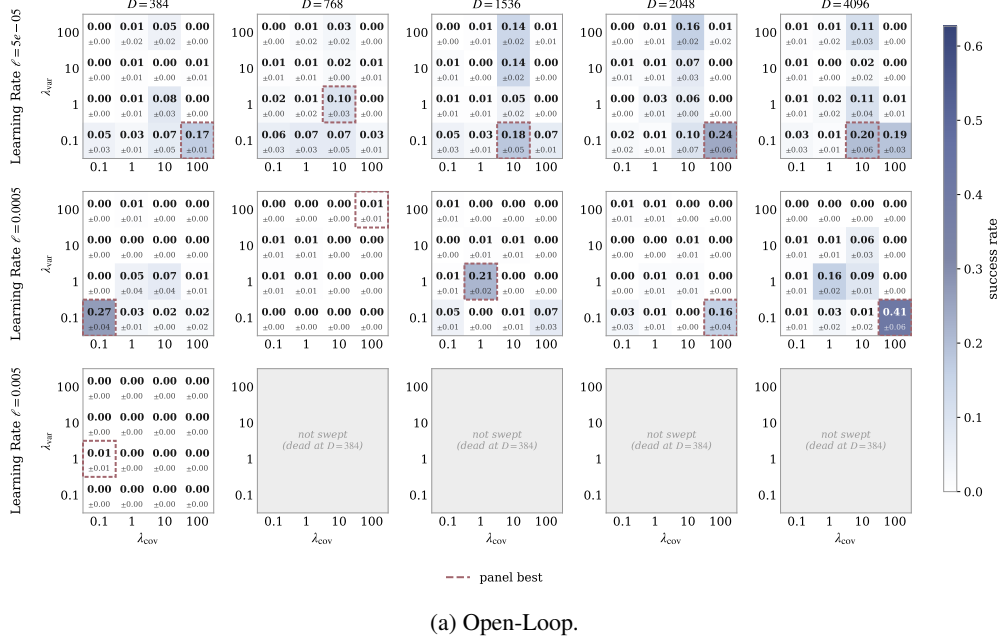
Dense-VICReg — MLP•LTV(k) — Closed-Loop plan success (mean $\pm$ std, 3 seeds)



(b) Closed-Loop.

Figure 13: **MLP•LTV(k) Predictor.** Hyperparameter Sweeps for VICReg-based JEPA models in the PushT Environment.

Dense-VICReg — MLP•LTI(k) — Open-Loop plan success (mean $\pm$ std, 3 seeds)



Dense-VICReg — MLP•LTI(k) — Closed-Loop plan success (mean $\pm$ std, 3 seeds)

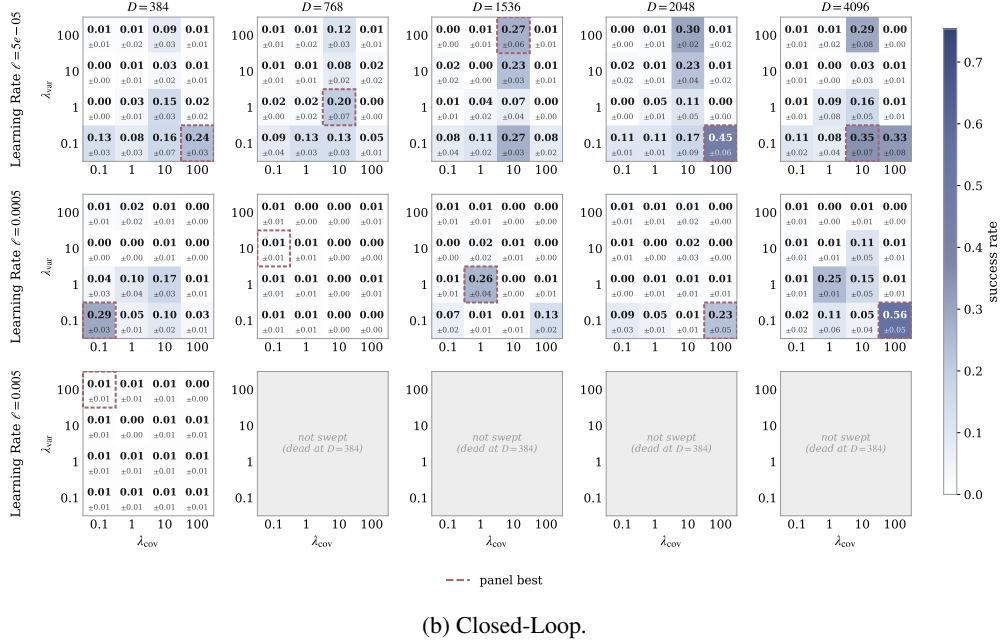


Figure 14: **MLP•LTI(k) Predictor.** Hyperparameter Sweeps for VICReg-based JEPa models in the PushT Environment.